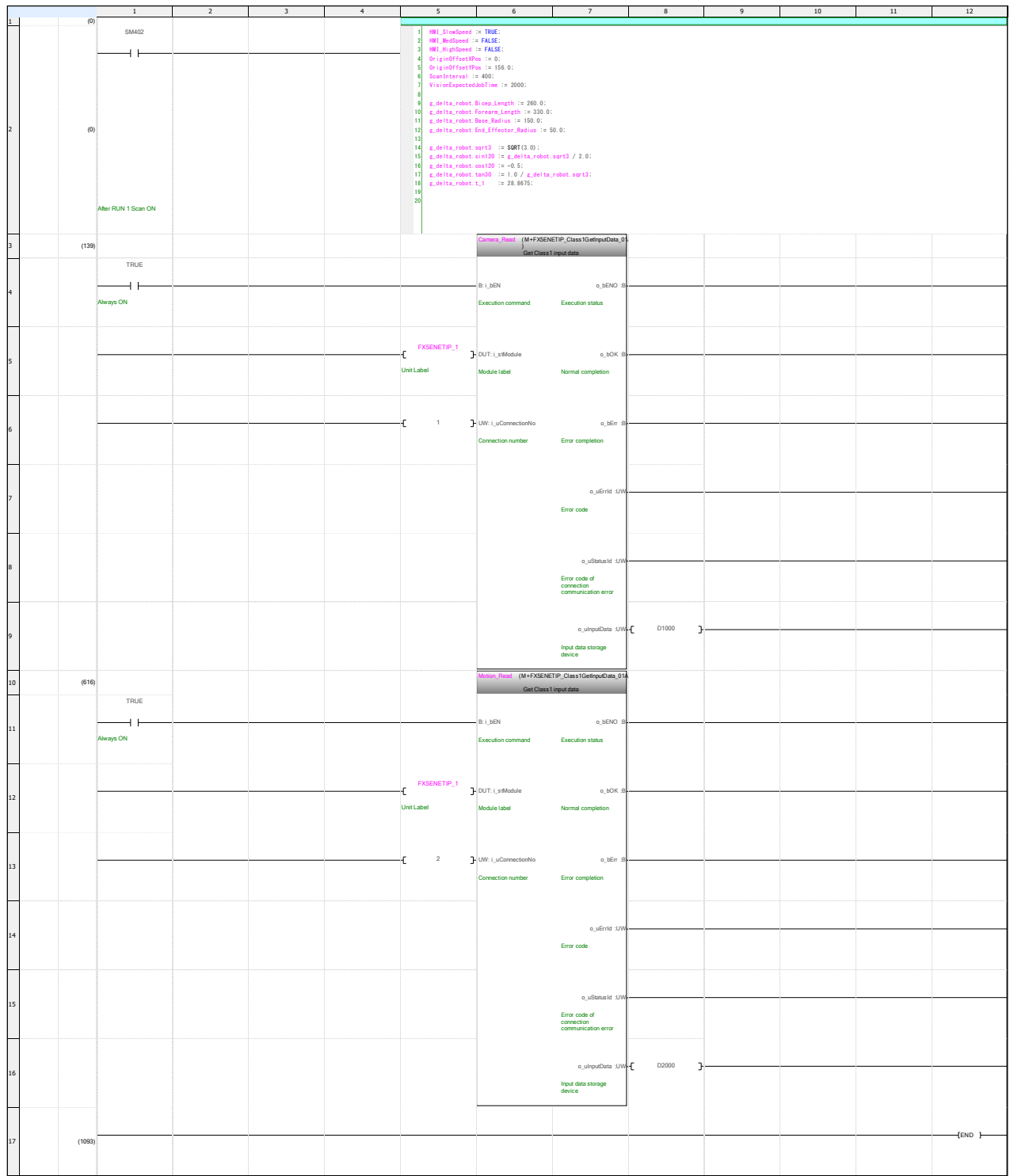




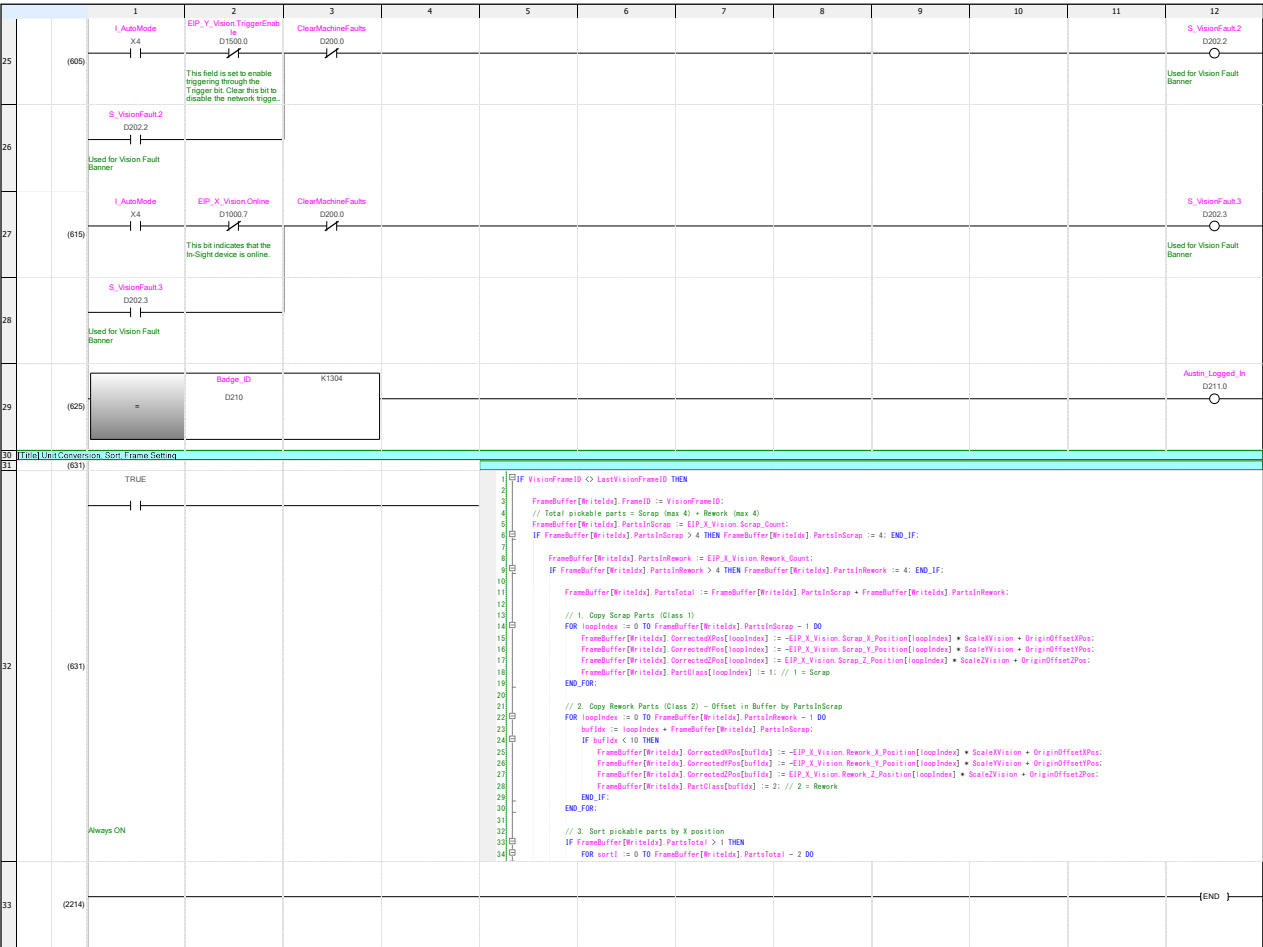
Inline Structured Text: EIP_Read (Row2)

```
1  HMI_SlowSpeed := TRUE;
2  HMI_MedSpeed  := FALSE;
3  HMI_HighSpeed  := FALSE;
4  OriginOffsetXPos := 0;
5  OriginOffsetYPos := 156.0;
6  ScanInterval   := 400;
7  VisionExpectedJobTime := 2000;
8
9  g_delta_robot.Bicep_Length := 260.0;
10 g_delta_robot.Forearm_Length := 330.0;
11 g_delta_robot.Base_Radius := 150.0;
12 g_delta_robot.End_Effector_Radius := 50.0;
13
14 g_delta_robot.sqrt3 := SQRT(3.0);
15 g_delta_robot.sin120 := g_delta_robot.sqrt3 / 2.0;
16 g_delta_robot.cos120 := -0.5;
17 g_delta_robot.tan30 := 1.0 / g_delta_robot.sqrt3;
18 g_delta_robot.t_1 := 28.8675;
19
20
```

		1	2	3	4	5	6	7	8	9	10	11	12
1	(0)						Camera_Write (M+FX5ENE P_Class1Se Set Class1 Output data						
2		TRUE Always ON					B: i_bEN Execution command	o_bENO :B Execution status					
3						FX5ENETI P_1 Unit Label	i_stMo DUT: dule Module label	o_bOK :B Normal completio n					
4						[1]	i_uCon UW: nectio... Connectio n number	o_bErr: B Error completio n					
5						[D1500]	i_uOutp UW: utData Output data storage device	o_uErrl d :UW Error code					
6								o_uStat usId :UW Error code of connectio n comm					
7	(477)						Motion_Write (M+FX5ENE _Class1SetO Set Class1 Output data						
8		TRUE Always ON					B: i_bEN Execution command	o_bENO :B Execution status					
9						FX5ENETI P_1 Unit Label	i_stMo DUT: dule Module label	o_bOK :B Normal completio n					
10						[2]	i_uCon UW: nectio... Connectio n number	o_bErr: B Error completio n					

		1	2	3	4	5	6	7	8	9	10	11	12
11						[D2500]	i_uOutp UW: utData	o_uErrl d :UW					
							Output data storage device	Error code					
12								o_uStat usid :UW					
								Error code of connectio n comm					
13	(954)												[END]



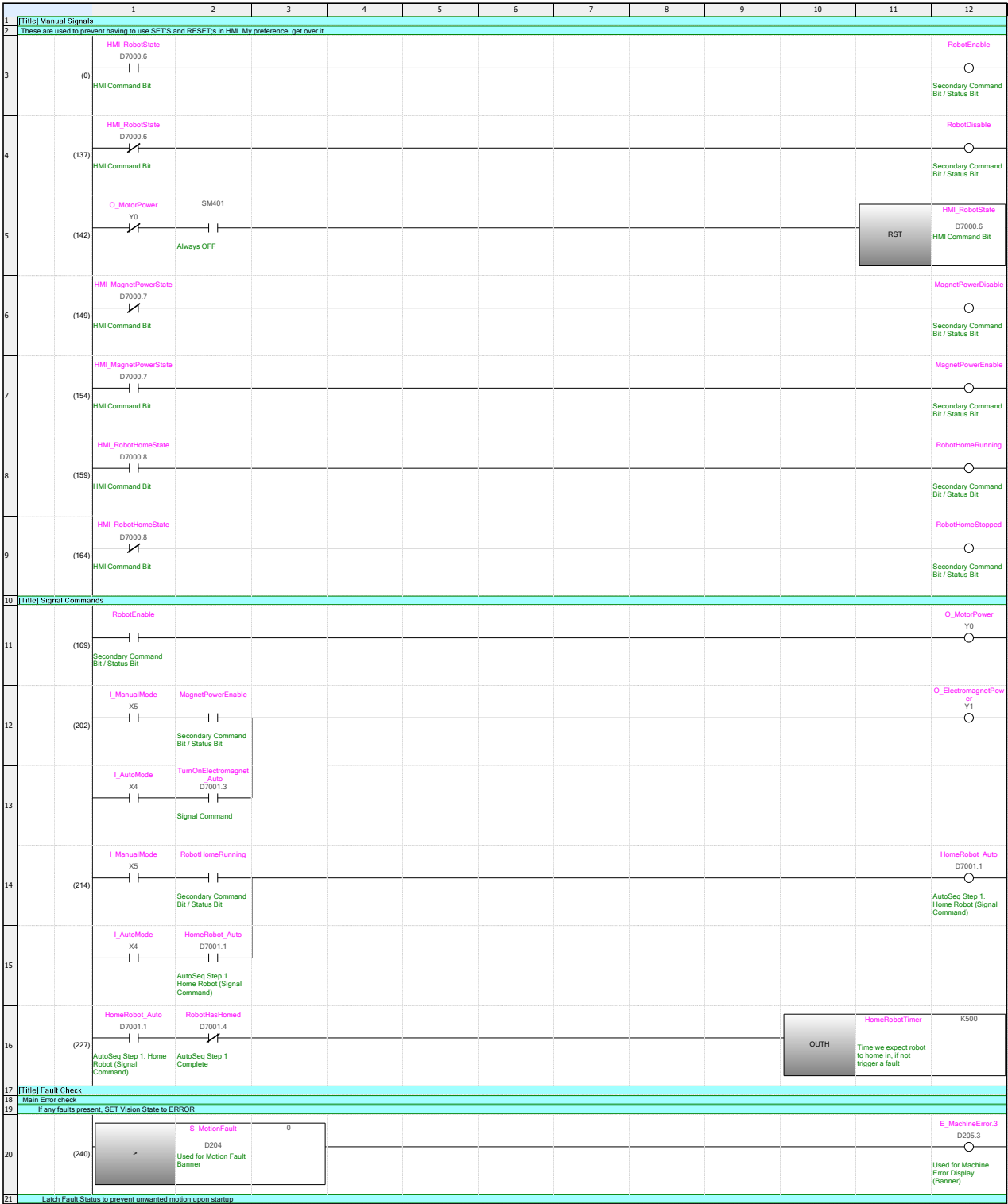


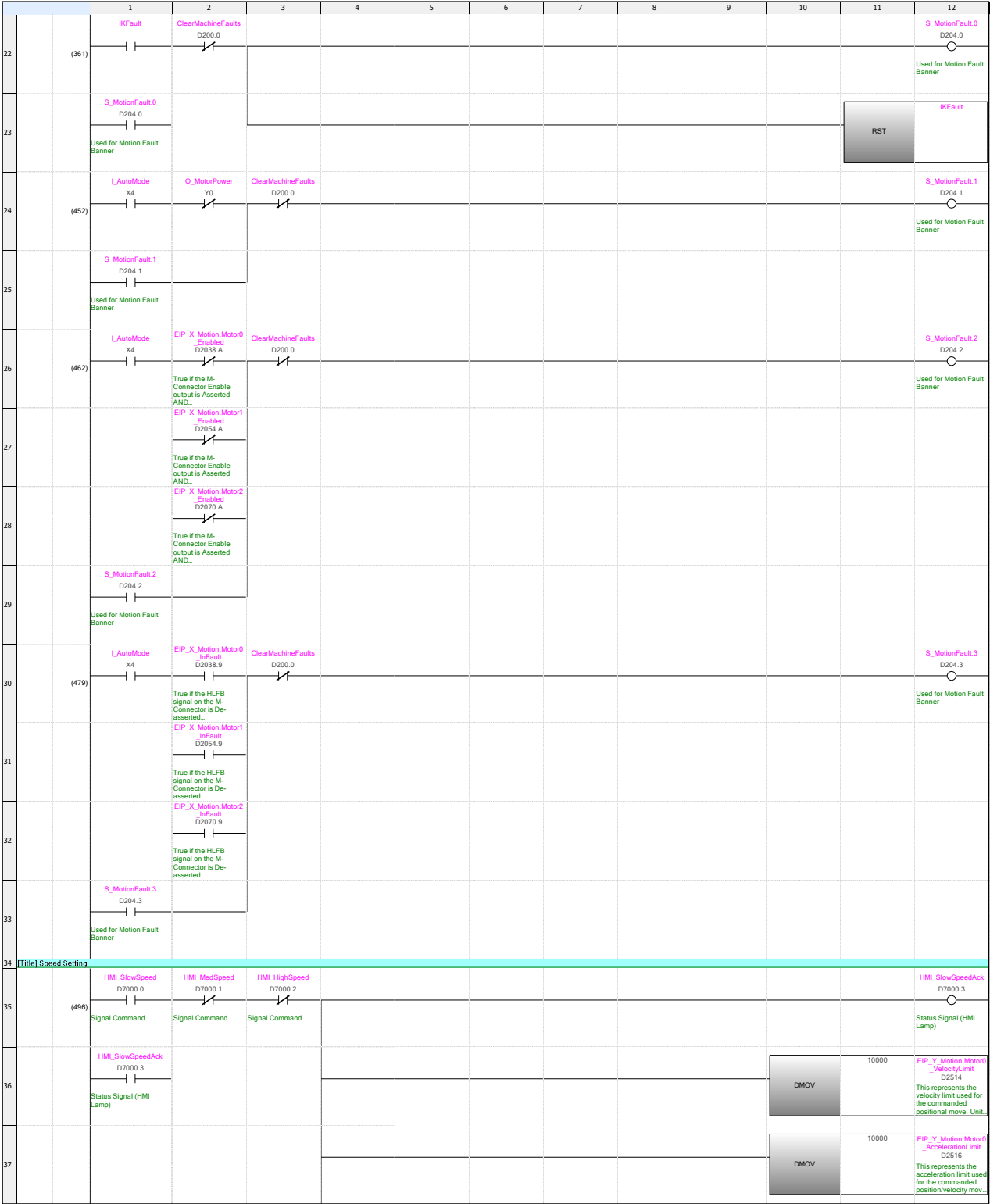
Inline Structured Text: Vision (Row32)

```
1 IF VisionFrameID <> LastVisionFrameID THEN
2
3     FrameBuffer[WriteIdx].FrameID := VisionFrameID;
4     // Total pickable parts = Scrap (max 4) + Rework (max 4)
5     FrameBuffer[WriteIdx].PartsInScrap := EIP_X_Vision.Scrap_Count;
6     IF FrameBuffer[WriteIdx].PartsInScrap > 4 THEN FrameBuffer[WriteIdx].PartsInScrap := 4; END_IF;
7
8     FrameBuffer[WriteIdx].PartsInRework := EIP_X_Vision.Rework_Count;
9     IF FrameBuffer[WriteIdx].PartsInRework > 4 THEN FrameBuffer[WriteIdx].PartsInRework := 4;
10    END_IF;
11
12    FrameBuffer[WriteIdx].PartsTotal := FrameBuffer[WriteIdx].PartsInScrap +
13    FrameBuffer[WriteIdx].PartsInRework;
14
15    // 1. Copy Scrap Parts (Class 1)
16    FOR loopIndex := 0 TO FrameBuffer[WriteIdx].PartsInScrap - 1 DO
17        FrameBuffer[WriteIdx].CorrectedXPos[loopIndex] :=
18        -EIP_X_Vision.Scrap_X_Position[loopIndex] * ScaleXVision + OriginOffsetXPos;
19        FrameBuffer[WriteIdx].CorrectedYPos[loopIndex] :=
20        -EIP_X_Vision.Scrap_Y_Position[loopIndex] * ScaleYVision + OriginOffsetYPos;
21        FrameBuffer[WriteIdx].CorrectedZPos[loopIndex] :=
22        EIP_X_Vision.Scrap_Z_Position[loopIndex] * ScaleZVision + OriginOffsetZPos;
23        FrameBuffer[WriteIdx].PartClass[loopIndex] := 1; // 1 = Scrap
24    END_FOR;
25
26    // 2. Copy Rework Parts (Class 2) - Offset in Buffer by PartsInScrap
27    FOR loopIndex := 0 TO FrameBuffer[WriteIdx].PartsInRework - 1 DO
28        bufIdx := loopIndex + FrameBuffer[WriteIdx].PartsInScrap;
29        IF bufIdx < 10 THEN
30            FrameBuffer[WriteIdx].CorrectedXPos[bufIdx] :=
31            -EIP_X_Vision.Rework_X_Position[loopIndex] * ScaleXVision + OriginOffsetXPos;
32            FrameBuffer[WriteIdx].CorrectedYPos[bufIdx] :=
33            -EIP_X_Vision.Rework_Y_Position[loopIndex] * ScaleYVision + OriginOffsetYPos;
34            FrameBuffer[WriteIdx].CorrectedZPos[bufIdx] :=
35            EIP_X_Vision.Rework_Z_Position[loopIndex] * ScaleZVision + OriginOffsetZPos;
36            FrameBuffer[WriteIdx].PartClass[bufIdx] := 2; // 2 = Rework
37        END_IF;
38    END_FOR;
39
40    // 3. Sort pickable parts by X position
41    IF FrameBuffer[WriteIdx].PartsTotal > 1 THEN
42        FOR sortI := 0 TO FrameBuffer[WriteIdx].PartsTotal - 2 DO
43            FOR sortJ := 0 TO FrameBuffer[WriteIdx].PartsTotal - sortI - 2 DO
44                IF FrameBuffer[WriteIdx].CorrectedXPos[sortJ] <
45                FrameBuffer[WriteIdx].CorrectedXPos[sortJ + 1] THEN
46                    // Swap X
```

Inline Structured Text: Vision (Row32)

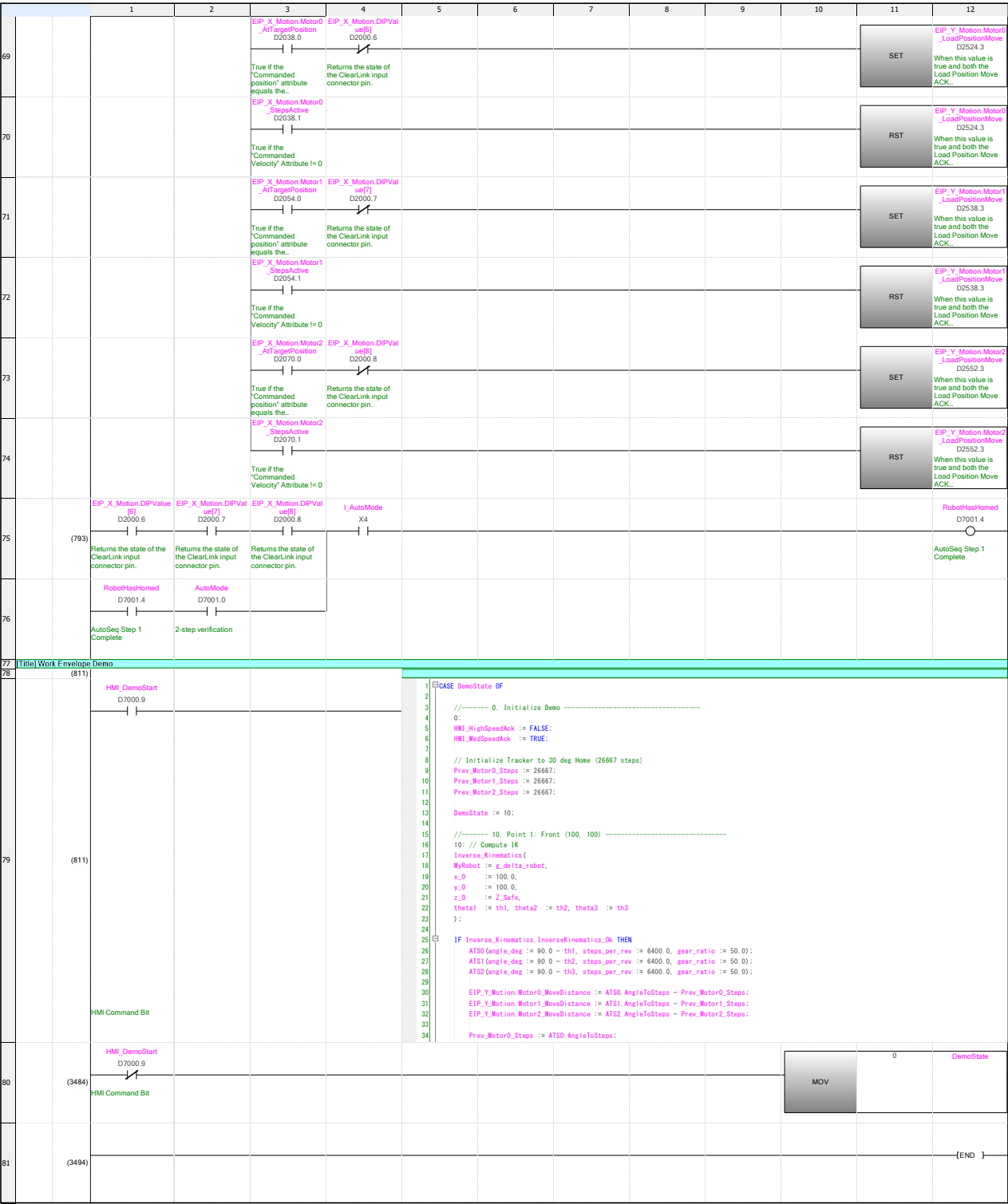
```
38             tempReal := FrameBuffer[WriteIdx].CorrectedXPos[sortJ];
39             FrameBuffer[WriteIdx].CorrectedXPos[sortJ] :=
FrameBuffer[WriteIdx].CorrectedXPos[sortJ + 1];
40             FrameBuffer[WriteIdx].CorrectedXPos[sortJ + 1] := tempReal;
41
42             // Swap Y
43             tempReal := FrameBuffer[WriteIdx].CorrectedYPos[sortJ];
44             FrameBuffer[WriteIdx].CorrectedYPos[sortJ] :=
FrameBuffer[WriteIdx].CorrectedYPos[sortJ + 1];
45             FrameBuffer[WriteIdx].CorrectedYPos[sortJ + 1] := tempReal;
46
47             // Swap Z
48             tempReal := FrameBuffer[WriteIdx].CorrectedZPos[sortJ];
49             FrameBuffer[WriteIdx].CorrectedZPos[sortJ] :=
FrameBuffer[WriteIdx].CorrectedZPos[sortJ + 1];
50             FrameBuffer[WriteIdx].CorrectedZPos[sortJ + 1] := tempReal;
51
52             // Swap Class ID
53             tempInt := FrameBuffer[WriteIdx].PartClass[sortJ];
54             FrameBuffer[WriteIdx].PartClass[sortJ] :=
FrameBuffer[WriteIdx].PartClass[sortJ + 1];
55             FrameBuffer[WriteIdx].PartClass[sortJ + 1] := tempInt;
56             END_IF;
57             END_FOR;
58             END_FOR;
59             END_IF;
60
61             // Advance write pointer
62             WriteIdx := (WriteIdx + 1) MOD 6;
63             IF FrameCount < 5 THEN
64                 FrameCount := FrameCount + 1;
65             END_IF;
66
67             LastVisionFrameID := VisionFrameID;
68             END_IF;
69
```



	1	2	3	4	5	6	7	8	9	10	11	12
38										DMOV	10000	EIP_Y_Motion.Motor0 _JogVelocity D2520 This represents the target velocity of the next commanded velocity move. Units...
39										DMOV	10000	EIP_Y_Motion.Motor1 _VelocityLimit D2528 This represents the velocity limit used for the commanded positional move. Unit...
40										DMOV	10000	EIP_Y_Motion.Motor1 _AccelerationLimit D2530 This represents the acceleration limit used for the commanded position/velocity mov...
41										DMOV	10000	EIP_Y_Motion.Motor1 _JogVelocity D2534 This represents the target velocity of the next commanded velocity move. Units...
42										DMOV	10000	EIP_Y_Motion.Motor2 _VelocityLimit D2542 This represents the velocity limit used for the commanded positional move. Unit...
43										DMOV	10000	EIP_Y_Motion.Motor2 _AccelerationLimit D2544 This represents the acceleration limit used for the commanded position/velocity mov...
44										DMOV	10000	EIP_Y_Motion.Motor2 _JogVelocity D2548 This represents the target velocity of the next commanded velocity move. Units...
45	(S83) HMI_MedSpeed D7000.1 Signal Command	HMI_SlowSpeed D7000.0 Signal Command	HMI_HighSpeed D7000.2 Signal Command									HMI_MedSpeedAck D7000.4 Status Signal (HMI Lamp)
46	HMI_MedSpeedAck D7000.4 Status Signal (HMI Lamp)									DMOV	10000	EIP_Y_Motion.Motor0 _VelocityLimit D2514 This represents the velocity limit used for the commanded positional move. Unit...
47										DMOV	100000	EIP_Y_Motion.Motor0 _AccelerationLimit D2516 This represents the acceleration limit used for the commanded position/velocity mov...
48										DMOV	10000	EIP_Y_Motion.Motor0 _JogVelocity D2520 This represents the target velocity of the next commanded velocity move. Units...
49										DMOV	10000	EIP_Y_Motion.Motor1 _VelocityLimit D2528 This represents the velocity limit used for the commanded positional move. Unit...
50										DMOV	100000	EIP_Y_Motion.Motor1 _AccelerationLimit D2530 This represents the acceleration limit used for the commanded position/velocity mov...
51										DMOV	10000	EIP_Y_Motion.Motor1 _JogVelocity D2534 This represents the target velocity of the next commanded velocity move. Units...
52										DMOV	10000	EIP_Y_Motion.Motor2 _VelocityLimit D2542 This represents the velocity limit used for the commanded positional move. Unit...

	1	2	3	4	5	6	7	8	9	10	11	12
53										DMOV	100000	EIP_Y_MotionMotor2_AccelerationLimit D2544 This represents the acceleration limit used for the commanded position/velocity move.
54										DMOV	100000	EIP_Y_MotionMotor2_JogVelocity D2548 This represents the target velocity of the next commanded velocity move. Units.
55	(643) HMI_HighSpeed D7000.2 Signal Command	HMI_MedSpeed D7000.1 Signal Command	HMI_SlowSpeed D7000.0 Signal Command									HMI_HighSpeedAck D7000.5 Status Signal (HMI Lamp)
56	HMI_HighSpeedAck D7000.5 Status Signal (HMI Lamp)									DMOV	200000	EIP_Y_MotionMotor0_VelocityLimit D2514 This represents the velocity limit used for the commanded positional move. Unit.
57										DMOV	300000	EIP_Y_MotionMotor0_AccelerationLimit D2516 This represents the acceleration limit used for the commanded position/velocity move.
58										DMOV	200000	EIP_Y_MotionMotor0_JogVelocity D2520 This represents the target velocity of the next commanded velocity move. Units.
59										DMOV	200000	EIP_Y_MotionMotor1_VelocityLimit D2528 This represents the velocity limit used for the commanded positional move. Unit.
60										DMOV	320000	EIP_Y_MotionMotor1_AccelerationLimit D2530 This represents the acceleration limit used for the commanded position/velocity move.
61										DMOV	200000	EIP_Y_MotionMotor1_JogVelocity D2534 This represents the target velocity of the next commanded velocity move. Units.
62										DMOV	200000	EIP_Y_MotionMotor2_VelocityLimit D2542 This represents the velocity limit used for the commanded positional move. Unit.
63										DMOV	320000	EIP_Y_MotionMotor2_AccelerationLimit D2544 This represents the acceleration limit used for the commanded position/velocity move.
64										DMOV	200000	EIP_Y_MotionMotor2_JogVelocity D2548 This represents the target velocity of the next commanded velocity move. Units.
65	Title Homing Seq											
66	(703) I_AutoMode X4	HomeRobot_Auto D7001.1 AutoSeq Step 1, Home Robot (Signal Command)								DMOV	300	EIP_Y_MotionMotor0_MoveDistance D2512 This represents the target position of the next commanded positional move.
67										DMOV	300	EIP_Y_MotionMotor1_MoveDistance D2526 This represents the target position of the next commanded positional move.
68										DMOV	300	EIP_Y_MotionMotor2_MoveDistance D2540 This represents the target position of the next commanded positional move.



Inline Structured Text: Motion (Row79)

```

1 CASE DemoState OF
2
3     //----- 0. Initialize Demo -----
4     0:
5         HMI_HighSpeedAck := FALSE;
6         HMI_MedSpeedAck  := TRUE;
7
8         // Initialize Tracker to 30 deg Home (26667 steps)
9         Prev_Motor0_Steps := 26667;
10        Prev_Motor1_Steps := 26667;
11        Prev_Motor2_Steps := 26667;
12
13        DemoState := 10;
14
15        //----- 10. Point 1: Front (100, 100) -----
16        10: // Compute IK
17        Inverse_Kinematics(
18        MyRobot := g_delta_robot,
19        x_0     := 100.0,
20        y_0     := 100.0,
21        z_0     := Z_Safe,
22        theta1  := th1, theta2 := th2, theta3 := th3
23        );
24
25        IF Inverse_Kinematics.InverseKinematics_Ok THEN
26            ATSO(angle_deg := 90.0 - th1, steps_per_rev := 6400.0, gear_ratio := 50.0);
27            ATS1(angle_deg := 90.0 - th2, steps_per_rev := 6400.0, gear_ratio := 50.0);
28            ATS2(angle_deg := 90.0 - th3, steps_per_rev := 6400.0, gear_ratio := 50.0);
29
30            EIP_Y_Motion.Motor0_MoveDistance := ATSO.AngleToSteps - Prev_Motor0_Steps;
31            EIP_Y_Motion.Motor1_MoveDistance := ATS1.AngleToSteps - Prev_Motor1_Steps;
32            EIP_Y_Motion.Motor2_MoveDistance := ATS2.AngleToSteps - Prev_Motor2_Steps;
33
34            Prev_Motor0_Steps := ATSO.AngleToSteps;
35            Prev_Motor1_Steps := ATS1.AngleToSteps;
36            Prev_Motor2_Steps := ATS2.AngleToSteps;
37
38            EIP_Y_Motion.Motor0_LoadPositionMove := TRUE;
39            EIP_Y_Motion.Motor1_LoadPositionMove := TRUE;
40            EIP_Y_Motion.Motor2_LoadPositionMove := TRUE;
41            DemoState := 11;
42        END_IF;
43
44        11: // Wait Ack
45        IF EIP_X_Motion.Motor0_LoadPositionMoveAck AND EIP_X_Motion.Motor1_LoadPositionMoveAck AND
EIP_X_Motion.Motor2_LoadPositionMoveAck THEN

```

Inline Structured Text: Motion (Row79)

```

46     EIP_Y_Motion.Motor0_LoadPositionMove := FALSE;
47     EIP_Y_Motion.Motor1_LoadPositionMove := FALSE;
48     EIP_Y_Motion.Motor2_LoadPositionMove := FALSE;
49     DemoState := 12;
50 END_IF;
51
52 12: // Wait Ack Clear
53 IF NOT EIP_X_Motion.Motor0_LoadPositionMoveAck AND NOT EIP_X_Motion.Motor1_LoadPositionMoveAck AND
NOT EIP_X_Motion.Motor2_LoadPositionMoveAck THEN
54     DemoState := 13;
55 END_IF;
56
57 13: // Wait Completion
58 IF NOT EIP_X_Motion.Motor0_StepsActive AND EIP_X_Motion.Motor0_AtTargetPosition AND
59     NOT EIP_X_Motion.Motor1_StepsActive AND EIP_X_Motion.Motor1_AtTargetPosition AND
60     NOT EIP_X_Motion.Motor2_StepsActive AND EIP_X_Motion.Motor2_AtTargetPosition THEN
61     DemoState := 20;
62 END_IF;
63
64 //----- 20. Point 2: Right (-100,100) -----
65 20: // Compute IK
66 Inverse_Kinematics(
67 MyRobot := g_delta_robot,
68 x_0     := -100.0,
69 y_0     := 100.0,
70 z_0     := Z_Safe,
71 theta1  := th1, theta2 := th2, theta3 := th3
72 );
73
74 IF Inverse_Kinematics.InverseKinematics_Ok THEN
75     ATS0(angle_deg := 90.0 - th1, steps_per_rev := 6400.0, gear_ratio := 50.0);
76     ATS1(angle_deg := 90.0 - th2, steps_per_rev := 6400.0, gear_ratio := 50.0);
77     ATS2(angle_deg := 90.0 - th3, steps_per_rev := 6400.0, gear_ratio := 50.0);
78
79     EIP_Y_Motion.Motor0_MoveDistance := ATS0.AngleToSteps - Prev_Motor0_Steps;
80     EIP_Y_Motion.Motor1_MoveDistance := ATS1.AngleToSteps - Prev_Motor1_Steps;
81     EIP_Y_Motion.Motor2_MoveDistance := ATS2.AngleToSteps - Prev_Motor2_Steps;
82
83     Prev_Motor0_Steps := ATS0.AngleToSteps;
84     Prev_Motor1_Steps := ATS1.AngleToSteps;
85     Prev_Motor2_Steps := ATS2.AngleToSteps;
86
87     EIP_Y_Motion.Motor0_LoadPositionMove := TRUE;
88     EIP_Y_Motion.Motor1_LoadPositionMove := TRUE;
89     EIP_Y_Motion.Motor2_LoadPositionMove := TRUE;
90     DemoState := 21;

```

Inline Structured Text: Motion (Row79)

```

91     END_IF;
92
93     21: // Wait Ack
94     IF EIP_X_Motion.Motor0_LoadPositionMoveAck AND EIP_X_Motion.Motor1_LoadPositionMoveAck AND
EIP_X_Motion.Motor2_LoadPositionMoveAck THEN
95         EIP_Y_Motion.Motor0_LoadPositionMove := FALSE;
96         EIP_Y_Motion.Motor1_LoadPositionMove := FALSE;
97         EIP_Y_Motion.Motor2_LoadPositionMove := FALSE;
98         DemoState := 22;
99     END_IF;
100
101     22: // Wait Ack Clear
102     IF NOT EIP_X_Motion.Motor0_LoadPositionMoveAck AND NOT EIP_X_Motion.Motor1_LoadPositionMoveAck AND
NOT EIP_X_Motion.Motor2_LoadPositionMoveAck THEN
103         DemoState := 23;
104     END_IF;
105
106     23: // Wait Completion
107     IF NOT EIP_X_Motion.Motor0_StepsActive AND EIP_X_Motion.Motor0_AtTargetPosition AND
108         NOT EIP_X_Motion.Motor1_StepsActive AND EIP_X_Motion.Motor1_AtTargetPosition AND
109         NOT EIP_X_Motion.Motor2_StepsActive AND EIP_X_Motion.Motor2_AtTargetPosition THEN
110         DemoState := 30;
111     END_IF;
112
113     //----- 30. Point 3: Back (-100, -100) -----
114     30: // Compute IK
115     Inverse_Kinematics(
116     MyRobot := g_delta_robot,
117     x_0     := -100.0,
118     y_0     := -100.0,
119     z_0     := Z_Safe,
120     theta1  := th1, theta2 := th2, theta3 := th3
121     );
122
123     IF Inverse_Kinematics.InverseKinematics_Ok THEN
124         ATS0(angle_deg := 90.0 - th1, steps_per_rev := 6400.0, gear_ratio := 50.0);
125         ATS1(angle_deg := 90.0 - th2, steps_per_rev := 6400.0, gear_ratio := 50.0);
126         ATS2(angle_deg := 90.0 - th3, steps_per_rev := 6400.0, gear_ratio := 50.0);
127
128         EIP_Y_Motion.Motor0_MoveDistance := ATS0.AngleToSteps - Prev_Motor0_Steps;
129         EIP_Y_Motion.Motor1_MoveDistance := ATS1.AngleToSteps - Prev_Motor1_Steps;
130         EIP_Y_Motion.Motor2_MoveDistance := ATS2.AngleToSteps - Prev_Motor2_Steps;
131
132         Prev_Motor0_Steps := ATS0.AngleToSteps;
133         Prev_Motor1_Steps := ATS1.AngleToSteps;
134         Prev_Motor2_Steps := ATS2.AngleToSteps;

```


Inline Structured Text: Motion (Row79)

```

135
136     EIP_Y_Motion.Motor0_LoadPositionMove := TRUE;
137     EIP_Y_Motion.Motor1_LoadPositionMove := TRUE;
138     EIP_Y_Motion.Motor2_LoadPositionMove := TRUE;
139     DemoState := 31;
140 END_IF;
141
142 31: // Wait Ack
143 IF EIP_X_Motion.Motor0_LoadPositionMoveAck AND EIP_X_Motion.Motor1_LoadPositionMoveAck AND
EIP_X_Motion.Motor2_LoadPositionMoveAck THEN
144     EIP_Y_Motion.Motor0_LoadPositionMove := FALSE;
145     EIP_Y_Motion.Motor1_LoadPositionMove := FALSE;
146     EIP_Y_Motion.Motor2_LoadPositionMove := FALSE;
147     DemoState := 32;
148 END_IF;
149
150 32: // Wait Ack Clear
151 IF NOT EIP_X_Motion.Motor0_LoadPositionMoveAck AND NOT EIP_X_Motion.Motor1_LoadPositionMoveAck AND
NOT EIP_X_Motion.Motor2_LoadPositionMoveAck THEN
152     DemoState := 33;
153 END_IF;
154
155 33: // Wait Completion
156 IF NOT EIP_X_Motion.Motor0_StepsActive AND EIP_X_Motion.Motor0_AtTargetPosition AND
157     NOT EIP_X_Motion.Motor1_StepsActive AND EIP_X_Motion.Motor1_AtTargetPosition AND
158     NOT EIP_X_Motion.Motor2_StepsActive AND EIP_X_Motion.Motor2_AtTargetPosition THEN
159     DemoState := 40;
160 END_IF;
161
162 //----- 40. Point 4: Left (-100, 0) -----
163 40: // Compute IK
164 Inverse_Kinematics(
165     MyRobot := g_delta_robot,
166     x_0     := 100.0,
167     y_0     := -100.0,
168     z_0     := Z_Safe,
169     theta1  := th1, theta2 := th2, theta3 := th3
170 );
171
172 IF Inverse_Kinematics.InverseKinematics_0k THEN
173     ATS0(angle_deg := 90.0 - th1, steps_per_rev := 6400.0, gear_ratio := 50.0);
174     ATS1(angle_deg := 90.0 - th2, steps_per_rev := 6400.0, gear_ratio := 50.0);
175     ATS2(angle_deg := 90.0 - th3, steps_per_rev := 6400.0, gear_ratio := 50.0);
176
177     EIP_Y_Motion.Motor0_MoveDistance := ATS0.AngleToSteps - Prev_Motor0_Steps;
178     EIP_Y_Motion.Motor1_MoveDistance := ATS1.AngleToSteps - Prev_Motor1_Steps;

```

Inline Structured Text: Motion (Row79)

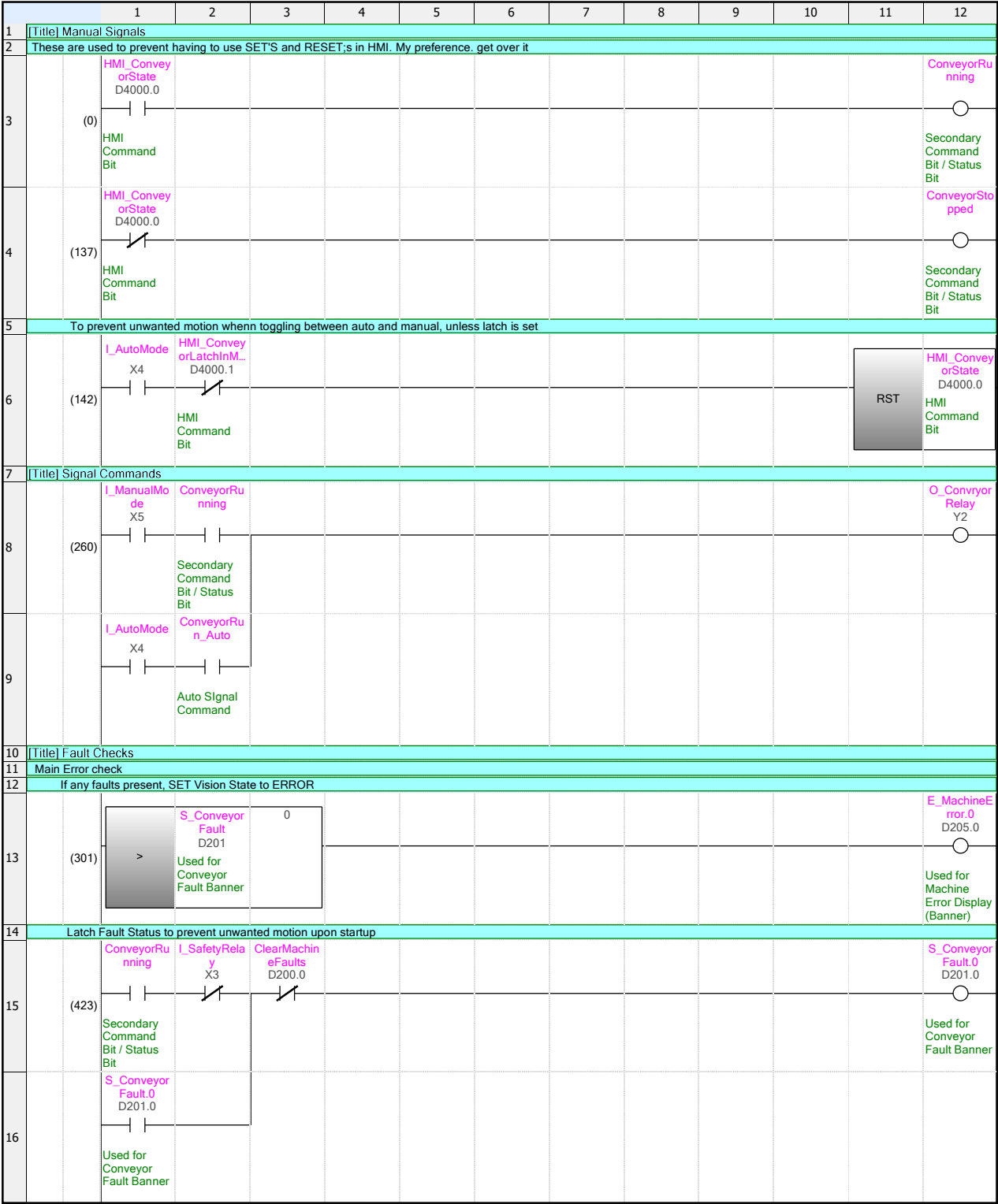
```

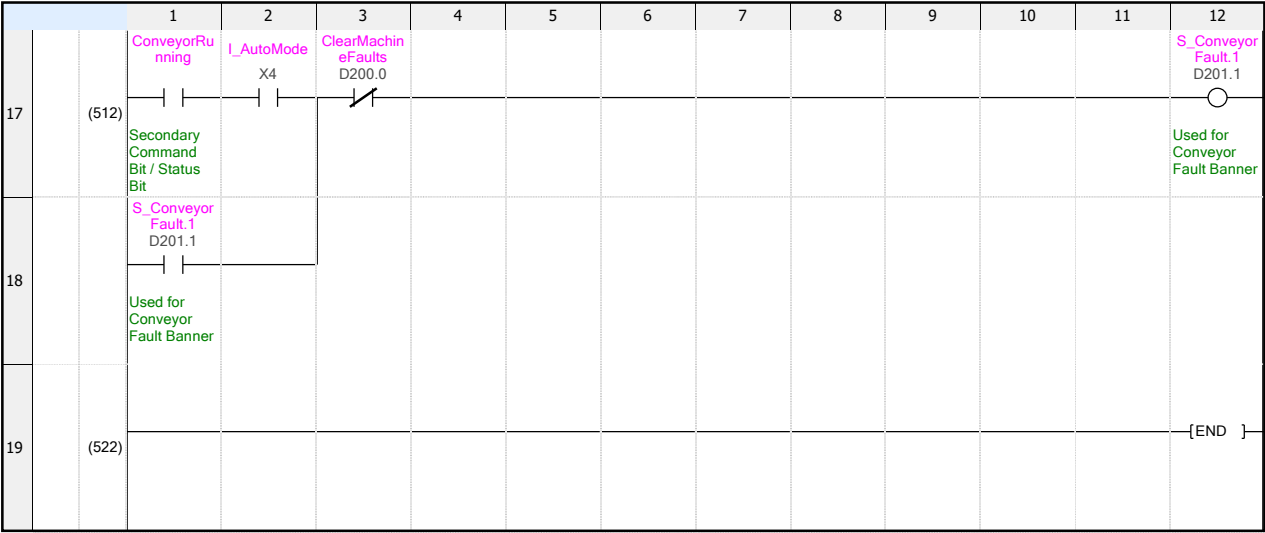
179         EIP_Y_Motion.Motor2_MoveDistance := ATS2.AngleToSteps - Prev_Motor2_Steps;
180
181         Prev_Motor0_Steps := ATS0.AngleToSteps;
182         Prev_Motor1_Steps := ATS1.AngleToSteps;
183         Prev_Motor2_Steps := ATS2.AngleToSteps;
184
185         EIP_Y_Motion.Motor0_LoadPositionMove := TRUE;
186         EIP_Y_Motion.Motor1_LoadPositionMove := TRUE;
187         EIP_Y_Motion.Motor2_LoadPositionMove := TRUE;
188         DemoState := 41;
189     END_IF;
190
191     41: // Wait Ack
192     IF EIP_X_Motion.Motor0_LoadPositionMoveAck AND EIP_X_Motion.Motor1_LoadPositionMoveAck AND
EIP_X_Motion.Motor2_LoadPositionMoveAck THEN
193         EIP_Y_Motion.Motor0_LoadPositionMove := FALSE;
194         EIP_Y_Motion.Motor1_LoadPositionMove := FALSE;
195         EIP_Y_Motion.Motor2_LoadPositionMove := FALSE;
196         DemoState := 42;
197     END_IF;
198
199     42: // Wait Ack Clear
200     IF NOT EIP_X_Motion.Motor0_LoadPositionMoveAck AND NOT EIP_X_Motion.Motor1_LoadPositionMoveAck AND
NOT EIP_X_Motion.Motor2_LoadPositionMoveAck THEN
201         DemoState := 43;
202     END_IF;
203
204     43: // Wait Completion
205     IF NOT EIP_X_Motion.Motor0_StepsActive AND EIP_X_Motion.Motor0_AtTargetPosition AND
206         NOT EIP_X_Motion.Motor1_StepsActive AND EIP_X_Motion.Motor1_AtTargetPosition AND
207         NOT EIP_X_Motion.Motor2_StepsActive AND EIP_X_Motion.Motor2_AtTargetPosition THEN
208         DemoState := 50;
209     END_IF;
210
211     //----- 50. Point 5: Center (0, 0) -----
212     50: // Compute IK
213     Inverse_Kinematics(
214     MyRobot := g_delta_robot,
215     x_0     := 0.0,
216     y_0     := 0.0,
217     z_0     := Z_Safe,
218     theta1  := th1, theta2 := th2, theta3 := th3
219     );
220
221     IF Inverse_Kinematics.InverseKinematics_0k THEN
222         ATS0(angle_deg := 90.0 - th1, steps_per_rev := 6400.0, gear_ratio := 50.0);

```

Inline Structured Text: Motion (Row79)

```
223     ATS1(angle_deg := 90.0 - th2, steps_per_rev := 6400.0, gear_ratio := 50.0);
224     ATS2(angle_deg := 90.0 - th3, steps_per_rev := 6400.0, gear_ratio := 50.0);
225
226     EIP_Y_Motion.Motor0_MoveDistance := ATS0.AngleToSteps - Prev_Motor0_Steps;
227     EIP_Y_Motion.Motor1_MoveDistance := ATS1.AngleToSteps - Prev_Motor1_Steps;
228     EIP_Y_Motion.Motor2_MoveDistance := ATS2.AngleToSteps - Prev_Motor2_Steps;
229
230     Prev_Motor0_Steps := ATS0.AngleToSteps;
231     Prev_Motor1_Steps := ATS1.AngleToSteps;
232     Prev_Motor2_Steps := ATS2.AngleToSteps;
233
234     EIP_Y_Motion.Motor0_LoadPositionMove := TRUE;
235     EIP_Y_Motion.Motor1_LoadPositionMove := TRUE;
236     EIP_Y_Motion.Motor2_LoadPositionMove := TRUE;
237     DemoState := 51;
238 END_IF;
239
240 51: // Wait Ack
241 IF EIP_X_Motion.Motor0_LoadPositionMoveAck AND EIP_X_Motion.Motor1_LoadPositionMoveAck AND
EIP_X_Motion.Motor2_LoadPositionMoveAck THEN
242     EIP_Y_Motion.Motor0_LoadPositionMove := FALSE;
243     EIP_Y_Motion.Motor1_LoadPositionMove := FALSE;
244     EIP_Y_Motion.Motor2_LoadPositionMove := FALSE;
245     DemoState := 52;
246 END_IF;
247
248 52: // Wait Ack Clear
249 IF NOT EIP_X_Motion.Motor0_LoadPositionMoveAck AND NOT EIP_X_Motion.Motor1_LoadPositionMoveAck AND
NOT EIP_X_Motion.Motor2_LoadPositionMoveAck THEN
250     DemoState := 53;
251 END_IF;
252
253 53: // Wait Completion
254 IF NOT EIP_X_Motion.Motor0_StepsActive AND EIP_X_Motion.Motor0_AtTargetPosition AND
255     NOT EIP_X_Motion.Motor1_StepsActive AND EIP_X_Motion.Motor1_AtTargetPosition AND
256     NOT EIP_X_Motion.Motor2_StepsActive AND EIP_X_Motion.Motor2_AtTargetPosition THEN
257     DemoState := 10; // Loop back to Point 1
258 END_IF;
259
260 END_CASE;
261
```





```
1 IF I_AutoMode AND I_SafetyRelay AND (E_MachineError = 0) THEN
2   AutoMode := TRUE;
3
4   //----- System Timer -----
5   IF HMI_BypassConveyor THEN
6     OUTH(NOT VisionScanTimer.S, VisionScanTimer, ScanInterval);
7   ELSE
8     OUTH(ConveyorRun_Auto AND NOT VisionScanTimer.S, VisionScanTimer,
ScanInterval);
9   END_IF;
10
11   IF VisionScanTimer.S THEN
12     AutoTriggerCamera := TRUE;
13     StartJobTimer := TRUE;
14     VisionScanTimer.N := 0;
15     VisionScanTimer.S := 0;
16   END_IF;
17   OUTH(StartJobTimer, VisionJobTimeCount, VisionExpectedJobTime);
18   IF VisionJobTimeCount.S THEN
19     StartJobTimer := FALSE;
20     VisionFrameID := VisionFrameID + 1;
21     VisionJobTimeCount.N := 0;
22     VisionJobTimeCount.S := 0;
23   END_IF;
24
25   CASE RobotState OF
26
27     0:
28       RobotState := 1;
29
30     //----- 1. Home Robot -----
31
32     1:
33       HomeRobot_Auto := TRUE;
34       HMI_HighSpeedAck := FALSE;
35       HMI_SlowSpeedAck := TRUE;
36       AutoMode := TRUE;
37       Homing_TurnOffElectromagnet := TRUE;
38       RobotState := 2;
39
40     //----- 2. Sync Tracker -----
41     2:
42       IF RobotHasHomed THEN
43         HomeRobot_Auto := FALSE;
44         HMI_HighSpeedAck := TRUE;
45         HMI_SlowSpeedAck := FALSE;
46         IF HMI_BypassConveyor THEN
```

```

46         ConveyorRun_Auto := FALSE;
47     ELSE
48         ConveyorRun_Auto := TRUE;
49     END_IF;
50
51     // Initialize Tracker to 30 deg Home (26667 steps)
52     Prev_Motor0_Steps := 26667;
53     Prev_Motor1_Steps := 26667;
54     Prev_Motor2_Steps := 26667;
55
56     RobotState := 10;    // Main Loop (Idle)
57 END_IF;
58
59 //----- 10. Main Loop (Idle)
-----
60 10:
61 IF FrameCount > 4 THEN
62     ActiveFrame := FrameBuffer[ReadIdx];
63     ReadIdx := (ReadIdx + 1) MOD 6;
64     FrameCount := FrameCount - 1;
65
66     loopIndex := 0;
67     PartsTotal := ActiveFrame.PartsTotal;
68
69     RobotState := 30;    // Step-Thru Main Loop
70 END_IF;
71
72 //----- 30. Step-Thru Main Loop
-----
73 30:
74 IF loopIndex < PartsTotal THEN
75     RobotState := 40;    // Compute IK for "Safe Pick"
76 ELSE
77     RobotState := 10;    // Main Loop (Idle)
78 END_IF;
79
80 //----- 40. Compute IK for "Safe Pick"
-----
81 40:
82
83     //IF loopIndex > 0 THEN
84     // Total Time = ((Full Intervals Passed) * ScanInterval) +
Current Cycle Time
85     CurrentTCP.X_Position := ActiveFrame.CorrectedXPos[loopIndex] +
(loopIndex * 20) + (BeltSpeed_mm_s * VisionScanTimer.N /
100); //(((VisionFrameID - 3) - ActiveFrame.FrameID) * ScanInterval +
VisionScanTimer.N) / 100);

```

```

86         //ELSE
87         //CurrentTCP.X_Position := ActiveFrame.CorrectedXPos[loopIndex];
88     //END_IF;
89
90     Inverse_Kinematics(
91     MyRobot := g_delta_robot,
92     x_0     := CurrentTCP.X_Position,
93     y_0     := ActiveFrame.CorrectedYPos[loopIndex],
94     z_0     := Z_Safe,
95     theta1  := th1, theta2 := th2, theta3 := th3
96     );
97
98     IF Inverse_Kinematics.InverseKinematics_Ok THEN
99         ATS0(angle_deg := 90.0 - th1, steps_per_rev := 6400.0,
gear_ratio := 50.0);
100        ATS1(angle_deg := 90.0 - th2, steps_per_rev := 6400.0,
gear_ratio := 50.0);
101        ATS2(angle_deg := 90.0 - th3, steps_per_rev := 6400.0,
gear_ratio := 50.0);
102
103        Motor0_Steps := ATS0.AngleToSteps - Prev_Motor0_Steps;
104        Motor1_Steps := ATS1.AngleToSteps - Prev_Motor1_Steps;
105        Motor2_Steps := ATS2.AngleToSteps - Prev_Motor2_Steps;
106
107        Prev_Motor0_Steps := ATS0.AngleToSteps;
108        Prev_Motor1_Steps := ATS1.AngleToSteps;
109        Prev_Motor2_Steps := ATS2.AngleToSteps;
110
111        IF ActiveFrame.CorrectedYPos[loopIndex] > 115 OR
ActiveFrame.CorrectedXPos[loopIndex] > 115 THEN
112            IKFault := TRUE;    // Error
113            ELSE
114                RobotState := 50;    // Move to "Safe Pick"
115            END_IF;
116
117        ELSE
118            RobotState := 300;    // Error
119        END_IF;
120
121        //----- 50. Move to "Safe Pick" -----
122        50: // Trigger Move (Ramp Start)
123        StartPos0 := EIP_X_Motion.Motor0_CommandedPosition;
124        StartPos1 := EIP_X_Motion.Motor1_CommandedPosition;
125        StartPos2 := EIP_X_Motion.Motor2_CommandedPosition;
126
127        EIP_Y_Motion.Motor0_MoveDistance := Motor0_Steps;
128        EIP_Y_Motion.Motor1_MoveDistance := Motor1_Steps;

```



```

129      EIP_Y_Motion.Motor2_MoveDistance := Motor2_Steps;
130      EIP_Y_Motion.Motor0_LoadPositionMove := TRUE;
131      EIP_Y_Motion.Motor1_LoadPositionMove := TRUE;
132      EIP_Y_Motion.Motor2_LoadPositionMove := TRUE;
133      RobotState := 51;
134
135      51: // Wait Ack
136      IF EIP_X_Motion.Motor0_LoadPositionMoveAck AND
EIP_X_Motion.Motor1_LoadPositionMoveAck AND
EIP_X_Motion.Motor2_LoadPositionMoveAck THEN
137          EIP_Y_Motion.Motor0_LoadPositionMove := FALSE;
138          EIP_Y_Motion.Motor1_LoadPositionMove := FALSE;
139          EIP_Y_Motion.Motor2_LoadPositionMove := FALSE;
140          RobotState := 52;
141      END_IF;
142
143      52: // Wait Ack Clear
144      IF NOT EIP_X_Motion.Motor0_LoadPositionMoveAck AND NOT
EIP_X_Motion.Motor1_LoadPositionMoveAck AND NOT
EIP_X_Motion.Motor2_LoadPositionMoveAck THEN
145          RobotState := 53;
146      END_IF;
147
148      53: // Ramp Monitor (Blend to Pick)
149      DistTotal := ABS(Motor0_Steps) + ABS(Motor1_Steps) +
ABS(Motor2_Steps);
150      DistDone := ABS(EIP_X_Motion.Motor0_CommandedPosition - StartPos0) +
151      ABS(EIP_X_Motion.Motor1_CommandedPosition - StartPos1) +
152      ABS(EIP_X_Motion.Motor2_CommandedPosition - StartPos2);
153
154      IF (DistTotal > 0 AND (DistDone * 100 / DistTotal) >= RampPercent)
OR DistTotal = 0 THEN
155          RobotState := 60; // Compute IK for "Actual Pick"
156      END_IF;
157
158      //----- 60. Compute IK for "Actual Pick"
-----
159      60:
160
161      //IF loopIndex > 0 THEN
162          CurrentTCP.X_Position := ActiveFrame.CorrectedXPos[loopIndex] +
(loopIndex * 5) + (BeltSpeed_mm_s * VisionScanTimer.N /
100); //(((VisionFrameID - 3) - ActiveFrame.FrameID) * ScanInterval +
VisionScanTimer.N) / 100);
163          //ELSE
164          //CurrentTCP.X_Position := ActiveFrame.CorrectedXPos[loopIndex];
165          //END_IF;

```

```

166
167     Inverse_Kinematics(
168     MyRobot := g_delta_robot,
169     x_0      := CurrentTCP.X_Position,
170     y_0      := ActiveFrame.CorrectedYPos[loopIndex],
171     z_0      := Z_Pick,
172     theta1   := th1, theta2 := th2, theta3 := th3
173     );
174
175     IF Inverse_Kinematics.InverseKinematics_Ok THEN
176         ATS0(angle_deg := 90.0 - th1, steps_per_rev := 6400.0,
177 gear_ratio := 50.0);
178         ATS1(angle_deg := 90.0 - th2, steps_per_rev := 6400.0,
179 gear_ratio := 50.0);
180         ATS2(angle_deg := 90.0 - th3, steps_per_rev := 6400.0,
181 gear_ratio := 50.0);
182
183         Motor0_Steps := ATS0.AngleToSteps - Prev_Motor0_Steps;
184         Motor1_Steps := ATS1.AngleToSteps - Prev_Motor1_Steps;
185         Motor2_Steps := ATS2.AngleToSteps - Prev_Motor2_Steps;
186
187         Prev_Motor0_Steps := ATS0.AngleToSteps;
188         Prev_Motor1_Steps := ATS1.AngleToSteps;
189         Prev_Motor2_Steps := ATS2.AngleToSteps;
190
191         RobotState := 70;    // Move to "Safe Pick"
192
193     ELSE
194         RobotState := 300;
195     END_IF;
196
197     //----- 70. Move to "Actual Pick" -----
198     70: // Trigger Move
199     EIP_Y_Motion.Motor0_MoveDistance := Motor0_Steps;
200     EIP_Y_Motion.Motor1_MoveDistance := Motor1_Steps;
201     EIP_Y_Motion.Motor2_MoveDistance := Motor2_Steps;
202     EIP_Y_Motion.Motor0_LoadPositionMove := TRUE;
203     EIP_Y_Motion.Motor1_LoadPositionMove := TRUE;
204     EIP_Y_Motion.Motor2_LoadPositionMove := TRUE;
205     RobotState := 71;
206
207     71: // Wait Ack
208     IF EIP_X_Motion.Motor0_LoadPositionMoveAck AND
209 EIP_X_Motion.Motor1_LoadPositionMoveAck AND
210 EIP_X_Motion.Motor2_LoadPositionMoveAck THEN
211         EIP_Y_Motion.Motor0_LoadPositionMove := FALSE;
212         EIP_Y_Motion.Motor1_LoadPositionMove := FALSE;

```

```

208         EIP_Y_Motion.Motor2_LoadPositionMove := FALSE;
209         RobotState := 72;
210     END_IF;
211
212     72: // Wait Ack Clear
213     IF NOT EIP_X_Motion.Motor0_LoadPositionMoveAck AND NOT
EIP_X_Motion.Motor1_LoadPositionMoveAck AND NOT
EIP_X_Motion.Motor2_LoadPositionMoveAck THEN
214         RobotState := 73;
215     END_IF;
216
217     73: // Wait Completion (Full Stop for Pick)
218     IF NOT EIP_X_Motion.Motor0_StepsActive AND
EIP_X_Motion.Motor0_AtTargetPosition AND
EIP_X_Motion.Motor1_AtTargetPosition AND
EIP_X_Motion.Motor2_AtTargetPosition THEN
219         RobotState := 80; // Pick Part
220     END_IF;
221
222     //----- 80. Pick Part
-----
223     80:
224     PickComplete := TRUE;
225     TurnOnElectromagnet_Auto := TRUE;
226     RobotState := 90; // Compute IK for "Safe Lift"
227
228     //----- 90. Compute IK for "Safe Lift"
-----
229     90:
230
231     //IF loopIndex > 0 THEN
232     // CurrentTCP.X_Position := ActiveFrame.CorrectedXPos[loopIndex] +
(BeltSpeed_mm_s * (((VisionFrameID - 4) - ActiveFrame.FrameID) *
ScanInterval + VisionScanTimer.N) / 100);
233     // ELSE
234     // CurrentTCP.X_Position := ActiveFrame.CorrectedXPos[loopIndex];
235     //END_IF;
236
237     Inverse_Kinematics(
238     MyRobot := g_delta_robot,
239     x_0 := CurrentTCP.X_Position,
240     y_0 := ActiveFrame.CorrectedYPos[loopIndex],
241     z_0 := Z_Safe,
242     theta1 := th1, theta2 := th2, theta3 := th3
243     );
244
245     IF Inverse_Kinematics.InverseKinematics_Ok THEN

```

```

246         ATS0(angle_deg := 90.0 - th1, steps_per_rev := 6400.0,
gear_ratio := 50.0);
247         ATS1(angle_deg := 90.0 - th2, steps_per_rev := 6400.0,
gear_ratio := 50.0);
248         ATS2(angle_deg := 90.0 - th3, steps_per_rev := 6400.0,
gear_ratio := 50.0);
249
250         Motor0_Steps := ATS0.AngleToSteps - Prev_Motor0_Steps;
251         Motor1_Steps := ATS1.AngleToSteps - Prev_Motor1_Steps;
252         Motor2_Steps := ATS2.AngleToSteps - Prev_Motor2_Steps;
253
254         Prev_Motor0_Steps := ATS0.AngleToSteps;
255         Prev_Motor1_Steps := ATS1.AngleToSteps;
256         Prev_Motor2_Steps := ATS2.AngleToSteps;
257
258         RobotState := 100; // Move to "Safe Lift"
259         ELSE
260         RobotState := 300;
261     END_IF;
262
263     //----- 100. Move to "Safe Lift" -----
264     100: // Trigger Move (Ramp Start)
265     StartPos0 := EIP_X_Motion.Motor0_CommandedPosition;
266     StartPos1 := EIP_X_Motion.Motor1_CommandedPosition;
267     StartPos2 := EIP_X_Motion.Motor2_CommandedPosition;
268
269     EIP_Y_Motion.Motor0_MoveDistance := Motor0_Steps;
270     EIP_Y_Motion.Motor1_MoveDistance := Motor1_Steps;
271     EIP_Y_Motion.Motor2_MoveDistance := Motor2_Steps;
272     EIP_Y_Motion.Motor0_LoadPositionMove := TRUE;
273     EIP_Y_Motion.Motor1_LoadPositionMove := TRUE;
274     EIP_Y_Motion.Motor2_LoadPositionMove := TRUE;
275     RobotState := 101;
276
277     101: // Wait Ack
278     IF EIP_X_Motion.Motor0_LoadPositionMoveAck AND
EIP_X_Motion.Motor1_LoadPositionMoveAck AND
EIP_X_Motion.Motor2_LoadPositionMoveAck THEN
279         EIP_Y_Motion.Motor0_LoadPositionMove := FALSE;
280         EIP_Y_Motion.Motor1_LoadPositionMove := FALSE;
281         EIP_Y_Motion.Motor2_LoadPositionMove := FALSE;
282         RobotState := 102;
283     END_IF;
284
285     102: // Wait Ack Clear
286     IF NOT EIP_X_Motion.Motor0_LoadPositionMoveAck AND NOT
EIP_X_Motion.Motor1_LoadPositionMoveAck AND NOT

```

```

EIP_X_Motion.Motor2_LoadPositionMoveAck THEN
287     RobotState := 103;
288     END_IF;
289
290     103: // Ramp Monitor (Blend to Intermediate)
291     DistTotal := ABS(Motor0_Steps) + ABS(Motor1_Steps) +
ABS(Motor2_Steps);
292     DistDone := ABS(EIP_X_Motion.Motor0_CommandedPosition - StartPos0) +
293     ABS(EIP_X_Motion.Motor1_CommandedPosition - StartPos1) +
294     ABS(EIP_X_Motion.Motor2_CommandedPosition - StartPos2);
295
296     IF (DistTotal > 0 AND (DistDone * 100 / DistTotal) >= RampPercent)
OR DistTotal = 0 THEN
297         RobotState := 110; // Compute IK for Intermediate Waypoint
298         END_IF;
299
300         //----- 110. Compute IK for Intermediate Waypoint
-----
301         110:
302
303         //IF loopIndex > 0 THEN
304         // CurrentTCP.X_Position := ActiveFrame.CorrectedXPos[loopIndex] +
(BeltSpeed_mm_s * (((VisionFrameID - 4) - ActiveFrame.FrameID) *
ScanInterval + VisionScanTimer.N) / 100);
305         // ELSE
306         // CurrentTCP.X_Position := ActiveFrame.CorrectedXPos[loopIndex];
307         //END_IF;
308
309         IF ActiveFrame.PartClass[loopIndex] = 1 THEN TargetX := 30.0;
ELSE TargetX := -30.0; END_IF;
310
311         Inverse_Kinematics(
312         MyRobot := g_delta_robot,
313         x_0 := (CurrentTCP.X_Position + TargetX) / 2.0,
314         y_0 := (ActiveFrame.CorrectedYPos[loopIndex] - 120.0) / 2.0,
315         z_0 := Z_Safe,
316         theta1 := th1, theta2 := th2, theta3 := th3
317         );
318
319         IF Inverse_Kinematics.InverseKinematics_Ok THEN
320             ATSO(angle_deg := 90.0 - th1, steps_per_rev := 6400.0,
gear_ratio := 50.0);
321             ATS1(angle_deg := 90.0 - th2, steps_per_rev := 6400.0,
gear_ratio := 50.0);
322             ATS2(angle_deg := 90.0 - th3, steps_per_rev := 6400.0,
gear_ratio := 50.0);
323

```

```
324         Motor0_Steps := ATS0.AngleToSteps - Prev_Motor0_Steps;
325         Motor1_Steps := ATS1.AngleToSteps - Prev_Motor1_Steps;
326         Motor2_Steps := ATS2.AngleToSteps - Prev_Motor2_Steps;
327
328         Prev_Motor0_Steps := ATS0.AngleToSteps;
329         Prev_Motor1_Steps := ATS1.AngleToSteps;
330         Prev_Motor2_Steps := ATS2.AngleToSteps;
331
332         RobotState := 120; // Move to Intermediate Waypoint
333     ELSE
334         RobotState := 300;
335 END_IF;
336
337 //----- 120. Move to Intermediate Waypoint
-----
338 120: // Trigger Move (Ramp Start)
339 StartPos0 := EIP_X_Motion.Motor0_CommandedPosition;
340 StartPos1 := EIP_X_Motion.Motor1_CommandedPosition;
341 StartPos2 := EIP_X_Motion.Motor2_CommandedPosition;
342
343 EIP_Y_Motion.Motor0_MoveDistance := Motor0_Steps;
344 EIP_Y_Motion.Motor1_MoveDistance := Motor1_Steps;
345 EIP_Y_Motion.Motor2_MoveDistance := Motor2_Steps;
346 EIP_Y_Motion.Motor0_LoadPositionMove := TRUE;
347 EIP_Y_Motion.Motor1_LoadPositionMove := TRUE;
348 EIP_Y_Motion.Motor2_LoadPositionMove := TRUE;
349 RobotState := 121;
350
351 121: // Wait Ack
352 IF EIP_X_Motion.Motor0_LoadPositionMoveAck AND
EIP_X_Motion.Motor1_LoadPositionMoveAck AND
EIP_X_Motion.Motor2_LoadPositionMoveAck THEN
353     EIP_Y_Motion.Motor0_LoadPositionMove := FALSE;
354     EIP_Y_Motion.Motor1_LoadPositionMove := FALSE;
355     EIP_Y_Motion.Motor2_LoadPositionMove := FALSE;
356     RobotState := 122;
357 END_IF;
358
359 122: // Wait Ack Clear
360 IF NOT EIP_X_Motion.Motor0_LoadPositionMoveAck AND NOT
EIP_X_Motion.Motor1_LoadPositionMoveAck AND NOT
EIP_X_Motion.Motor2_LoadPositionMoveAck THEN
361     RobotState := 123;
362 END_IF;
363
364 123: // Ramp Monitor (Blend to Safe Drop)
365 DistTotal := ABS(Motor0_Steps) + ABS(Motor1_Steps) +
```

```

ABS(Motor2_Steps);
366     DistDone := ABS(EIP_X_Motion.Motor0_CommandedPosition - StartPos0) +
367     ABS(EIP_X_Motion.Motor1_CommandedPosition - StartPos1) +
368     ABS(EIP_X_Motion.Motor2_CommandedPosition - StartPos2);
369
370     IF (DistTotal > 0 AND (DistDone * 100 / DistTotal) >= RampPercent)
OR DistTotal = 0 THEN
371         RobotState := 130; // Compute IK for Safe DropOff
372     END_IF;
373
374     //----- 130. Compute IK for Safe DropOff
-----
375     130:
376     PickComplete := FALSE;
377
378     IF ActiveFrame.PartClass[loopIndex] = 1 THEN // Scrap
379         Inverse_Kinematics(
380             MyRobot := g_delta_robot,
381             x_0 := 50.0,
382             y_0 := -120.0,
383             z_0 := Z_Safe,
384             theta1 := th1, theta2 := th2, theta3 := th3
385         );
386     ELSIF ActiveFrame.PartClass[loopIndex] = 2 THEN // Rework
387         Inverse_Kinematics(
388             MyRobot := g_delta_robot,
389             x_0 := -50.0,
390             y_0 := -120.0,
391             z_0 := Z_Safe,
392             theta1 := th1, theta2 := th2, theta3 := th3
393         );
394     ELSE
395         RobotState := 300; // Error: Unknown Class
396     END_IF;
397
398     IF Inverse_Kinematics.InverseKinematics_Ok AND RobotState = 130 THEN
399         ATS0(angle_deg := 90.0 - th1, steps_per_rev := 6400.0,
gear_ratio := 50.0);
400         ATS1(angle_deg := 90.0 - th2, steps_per_rev := 6400.0,
gear_ratio := 50.0);
401         ATS2(angle_deg := 90.0 - th3, steps_per_rev := 6400.0,
gear_ratio := 50.0);
402
403         Motor0_Steps := ATS0.AngleToSteps - Prev_Motor0_Steps;
404         Motor1_Steps := ATS1.AngleToSteps - Prev_Motor1_Steps;
405         Motor2_Steps := ATS2.AngleToSteps - Prev_Motor2_Steps;
406

```

```

407         Prev_Motor0_Steps := ATS0.AngleToSteps;
408         Prev_Motor1_Steps := ATS1.AngleToSteps;
409         Prev_Motor2_Steps := ATS2.AngleToSteps;
410
411         RobotState := 140; // Move to Safe DropOff
412     ELSE
413         RobotState := 300;
414 END_IF;
415
416 //----- 140. Move to Safe DropOff -----
417 140: // Trigger Move (Ramp Start)
418     StartPos0 := EIP_X_Motion.Motor0_CommandedPosition;
419     StartPos1 := EIP_X_Motion.Motor1_CommandedPosition;
420     StartPos2 := EIP_X_Motion.Motor2_CommandedPosition;
421
422     EIP_Y_Motion.Motor0_MoveDistance := Motor0_Steps;
423     EIP_Y_Motion.Motor1_MoveDistance := Motor1_Steps;
424     EIP_Y_Motion.Motor2_MoveDistance := Motor2_Steps;
425     EIP_Y_Motion.Motor0_LoadPositionMove := TRUE;
426     EIP_Y_Motion.Motor1_LoadPositionMove := TRUE;
427     EIP_Y_Motion.Motor2_LoadPositionMove := TRUE;
428     RobotState := 141;
429
430     141: // Wait Ack
431     IF EIP_X_Motion.Motor0_LoadPositionMoveAck AND
EIP_X_Motion.Motor1_LoadPositionMoveAck AND
EIP_X_Motion.Motor2_LoadPositionMoveAck THEN
432         EIP_Y_Motion.Motor0_LoadPositionMove := FALSE;
433         EIP_Y_Motion.Motor1_LoadPositionMove := FALSE;
434         EIP_Y_Motion.Motor2_LoadPositionMove := FALSE;
435         RobotState := 142;
436     END_IF;
437
438     142: // Wait Ack Clear
439     IF NOT EIP_X_Motion.Motor0_LoadPositionMoveAck AND NOT
EIP_X_Motion.Motor1_LoadPositionMoveAck AND NOT
EIP_X_Motion.Motor2_LoadPositionMoveAck THEN
440         RobotState := 143;
441     END_IF;
442
443     143: // Ramp Monitor (Blend to Drop)
444     DistTotal := ABS(Motor0_Steps) + ABS(Motor1_Steps) +
ABS(Motor2_Steps);
445     DistDone := ABS(EIP_X_Motion.Motor0_CommandedPosition - StartPos0) +
446     ABS(EIP_X_Motion.Motor1_CommandedPosition - StartPos1) +
447     ABS(EIP_X_Motion.Motor2_CommandedPosition - StartPos2);
448

```



```
449     IF (DistTotal > 0 AND (DistDone * 100 / DistTotal) >= RampPercent)
OR DistTotal = 0 THEN
450     RobotState := 150; // Compute IK for Actual DropOff
451     END_IF;
452
453     //----- 150. Compute IK for Actual DropOff
-----
454     150:
455     IF ActiveFrame.PartClass[loopIndex] = 1 THEN // Scrap
456         Inverse_Kinematics(
457             MyRobot := g_delta_robot,
458             x_0 := 50.0,
459             y_0 := -120.0,
460             z_0 := Z_Pick,
461             theta1 := th1, theta2 := th2, theta3 := th3
462         );
463     ELSIF ActiveFrame.PartClass[loopIndex] = 2 THEN // Rework
464         Inverse_Kinematics(
465             MyRobot := g_delta_robot,
466             x_0 := -50.0,
467             y_0 := -120.0,
468             z_0 := Z_Pick,
469             theta1 := th1, theta2 := th2, theta3 := th3
470         );
471     ELSE
472         RobotState := 300;
473     END_IF;
474
475     IF Inverse_Kinematics.InverseKinematics_Ok AND RobotState = 150 THEN
476         ATS0(angle_deg := 90.0 - th1, steps_per_rev := 6400.0,
gear_ratio := 50.0);
477         ATS1(angle_deg := 90.0 - th2, steps_per_rev := 6400.0,
gear_ratio := 50.0);
478         ATS2(angle_deg := 90.0 - th3, steps_per_rev := 6400.0,
gear_ratio := 50.0);
479
480         Motor0_Steps := ATS0.AngleToSteps - Prev_Motor0_Steps;
481         Motor1_Steps := ATS1.AngleToSteps - Prev_Motor1_Steps;
482         Motor2_Steps := ATS2.AngleToSteps - Prev_Motor2_Steps;
483
484         Prev_Motor0_Steps := ATS0.AngleToSteps;
485         Prev_Motor1_Steps := ATS1.AngleToSteps;
486         Prev_Motor2_Steps := ATS2.AngleToSteps;
487
488         RobotState := 160; // Move to Actual DropOff
489     ELSE
490         RobotState := 300;
```

```

491     END_IF;
492
493     //----- 160. Move to Actual DropOff
-----
494     160: // Trigger Move
495     EIP_Y_Motion.Motor0_MoveDistance := Motor0_Steps;
496     EIP_Y_Motion.Motor1_MoveDistance := Motor1_Steps;
497     EIP_Y_Motion.Motor2_MoveDistance := Motor2_Steps;
498     EIP_Y_Motion.Motor0_LoadPositionMove := TRUE;
499     EIP_Y_Motion.Motor1_LoadPositionMove := TRUE;
500     EIP_Y_Motion.Motor2_LoadPositionMove := TRUE;
501     RobotState := 161;
502
503     161: // Wait Ack
504     IF EIP_X_Motion.Motor0_LoadPositionMoveAck AND
EIP_X_Motion.Motor1_LoadPositionMoveAck AND
EIP_X_Motion.Motor2_LoadPositionMoveAck THEN
505         EIP_Y_Motion.Motor0_LoadPositionMove := FALSE;
506         EIP_Y_Motion.Motor1_LoadPositionMove := FALSE;
507         EIP_Y_Motion.Motor2_LoadPositionMove := FALSE;
508         RobotState := 162;
509     END_IF;
510
511     162: // Wait Ack Clear
512     IF NOT EIP_X_Motion.Motor0_LoadPositionMoveAck AND NOT
EIP_X_Motion.Motor1_LoadPositionMoveAck AND NOT
EIP_X_Motion.Motor2_LoadPositionMoveAck THEN
513         RobotState := 163;
514     END_IF;
515
516     163: // Wait Completion (Full Stop for Drop)
517     IF NOT EIP_X_Motion.Motor0_StepsActive AND
EIP_X_Motion.Motor0_AtTargetPosition AND
EIP_X_Motion.Motor1_AtTargetPosition AND
EIP_X_Motion.Motor2_AtTargetPosition THEN
518         RobotState := 170; // Drop Part
519     END_IF;
520
521     //----- 170. Drop Part -----
522     170:
523     PlaceComplete := TRUE;
524     TurnOnElectromagnet_Auto := FALSE;
525     IF ActiveFrame.PartClass[loopIndex] = 1 THEN
526         CurrentTCP.X_Position := 50.0;
527     ELSE
528         CurrentTCP.X_Position := -50.0;
529     END_IF;

```

```
530     CurrentTCP.Y_Position := -120.0;
531     CurrentTCP.Z_Position := Z_Pick;
532     RobotState := 180; // Compute IK for Safe DropOff Lift
533
534     //----- 180. Compute IK for Safe DropOff Lift
535     -----
536     180:
537     IF ActiveFrame.PartClass[loopIndex] = 1 THEN // Scrap
538         Inverse_Kinematics(
539             MyRobot := g_delta_robot,
540             x_0     := 50.0,
541             y_0     := -120.0,
542             z_0     := Z_Safe,
543             theta1  := th1, theta2 := th2, theta3 := th3
544         );
545     ELSIF ActiveFrame.PartClass[loopIndex] = 2 THEN // Rework
546         Inverse_Kinematics(
547             MyRobot := g_delta_robot,
548             x_0     := -50.0,
549             y_0     := -120.0,
550             z_0     := Z_Safe,
551             theta1  := th1, theta2 := th2, theta3 := th3
552         );
553     ELSE
554         RobotState := 300;
555     END_IF;
556
557     IF Inverse_Kinematics.InverseKinematics_Ok AND RobotState = 180 THEN
558         ATSO(angle_deg := 90.0 - th1, steps_per_rev := 6400.0,
559             gear_ratio := 50.0);
560         ATS1(angle_deg := 90.0 - th2, steps_per_rev := 6400.0,
561             gear_ratio := 50.0);
562         ATS2(angle_deg := 90.0 - th3, steps_per_rev := 6400.0,
563             gear_ratio := 50.0);
564
565         Motor0_Steps := ATSO.AngleToSteps - Prev_Motor0_Steps;
566         Motor1_Steps := ATS1.AngleToSteps - Prev_Motor1_Steps;
567         Motor2_Steps := ATS2.AngleToSteps - Prev_Motor2_Steps;
568
569         Prev_Motor0_Steps := ATSO.AngleToSteps;
570         Prev_Motor1_Steps := ATS1.AngleToSteps;
571         Prev_Motor2_Steps := ATS2.AngleToSteps;
572
573         RobotState := 190; // Move to Safe DropOff Lift
574     ELSE
575         RobotState := 300;
576     END_IF;
```

```

573
574      //----- 190. Move to Safe DropOff Lift
-----
575      190: // Trigger Move (Ramp Start)
576      StartPos0 := EIP_X_Motion.Motor0_CommandedPosition;
577      StartPos1 := EIP_X_Motion.Motor1_CommandedPosition;
578      StartPos2 := EIP_X_Motion.Motor2_CommandedPosition;
579
580      EIP_Y_Motion.Motor0_MoveDistance := Motor0_Steps;
581      EIP_Y_Motion.Motor1_MoveDistance := Motor1_Steps;
582      EIP_Y_Motion.Motor2_MoveDistance := Motor2_Steps;
583      EIP_Y_Motion.Motor0_LoadPositionMove := TRUE;
584      EIP_Y_Motion.Motor1_LoadPositionMove := TRUE;
585      EIP_Y_Motion.Motor2_LoadPositionMove := TRUE;
586      RobotState := 191;
587
588      191: // Wait Ack
589      IF EIP_X_Motion.Motor0_LoadPositionMoveAck AND
EIP_X_Motion.Motor1_LoadPositionMoveAck AND
EIP_X_Motion.Motor2_LoadPositionMoveAck THEN
590          EIP_Y_Motion.Motor0_LoadPositionMove := FALSE;
591          EIP_Y_Motion.Motor1_LoadPositionMove := FALSE;
592          EIP_Y_Motion.Motor2_LoadPositionMove := FALSE;
593          RobotState := 192;
594      END_IF;
595
596      192: // Wait Ack Clear
597      IF NOT EIP_X_Motion.Motor0_LoadPositionMoveAck AND NOT
EIP_X_Motion.Motor1_LoadPositionMoveAck AND NOT
EIP_X_Motion.Motor2_LoadPositionMoveAck THEN
598          RobotState := 193;
599      END_IF;
600
601      193: // Ramp Monitor (Blend to Next)
602      DistTotal := ABS(Motor0_Steps) + ABS(Motor1_Steps) +
ABS(Motor2_Steps);
603      DistDone := ABS(EIP_X_Motion.Motor0_CommandedPosition - StartPos0) +
604      ABS(EIP_X_Motion.Motor1_CommandedPosition - StartPos1) +
605      ABS(EIP_X_Motion.Motor2_CommandedPosition - StartPos2);
606
607      IF (DistTotal > 0 AND (DistDone * 100 / DistTotal) >= RampPercent)
OR DistTotal = 0 THEN
608          RobotState := 200;
609      END_IF;
610
611      //----- 200. Return Home / Next Part
-----

```

```
612      200:
613      loopIndex := loopIndex + 1;
614      IF loopIndex >= PartsTotal THEN
615          // Frame Complete
616          RobotState := 210; // Absolute Move Home
617      ELSE
618          RobotState := 220;
619      END_IF;
620
621      //----- 210. Home Correction (Absolute Move)
622      -----
623      210:
624      // Wait for motors to be fully stopped before correction to prevent
        jumps
625      IF NOT EIP_X_Motion.Motor0_StepsActive AND
        EIP_X_Motion.Motor0_AtTargetPosition AND
626      NOT EIP_X_Motion.Motor1_StepsActive AND
        EIP_X_Motion.Motor1_AtTargetPosition AND
627      NOT EIP_X_Motion.Motor2_StepsActive AND
        EIP_X_Motion.Motor2_AtTargetPosition THEN
628          // Return to Home (26667 steps)
629          EIP_Y_Motion.Motor0_MoveDistance := 26667 - Prev_Motor0_Steps;
630          EIP_Y_Motion.Motor1_MoveDistance := 26667 - Prev_Motor1_Steps;
631          EIP_Y_Motion.Motor2_MoveDistance := 26667 - Prev_Motor2_Steps;
632
633          Prev_Motor0_Steps := 26667;
634          Prev_Motor1_Steps := 26667;
635          Prev_Motor2_Steps := 26667;
636
637          EIP_Y_Motion.Motor0_LoadPositionMove := TRUE;
638          EIP_Y_Motion.Motor1_LoadPositionMove := TRUE;
639          EIP_Y_Motion.Motor2_LoadPositionMove := TRUE;
640
641          MotionStarted := FALSE;
642          AllAtTarget := FALSE;
643          RobotState := 211;
644      END_IF;
645
646      211:
647      IF EIP_X_Motion.Motor0_LoadPositionMoveAck AND
        EIP_X_Motion.Motor1_LoadPositionMoveAck AND
        EIP_X_Motion.Motor2_LoadPositionMoveAck THEN
648          EIP_Y_Motion.Motor0_LoadPositionMove := FALSE;
649          EIP_Y_Motion.Motor1_LoadPositionMove := FALSE;
650          EIP_Y_Motion.Motor2_LoadPositionMove := FALSE;
651          RobotState := 212;
```

```
652         END_IF;
653
654         212:
655         IF NOT EIP_X_Motion.Motor0_LoadPositionMoveAck AND NOT
EIP_X_Motion.Motor1_LoadPositionMoveAck AND NOT
EIP_X_Motion.Motor2_LoadPositionMoveAck THEN
656             MotionStarted := TRUE;
657             RobotState := 213;
658         END_IF;
659
660         213:
661         IF NOT EIP_X_Motion.Motor0_StepsActive AND
EIP_X_Motion.Motor0_AtTargetPosition AND
EIP_X_Motion.Motor1_AtTargetPosition AND
EIP_X_Motion.Motor2_AtTargetPosition THEN
662             RobotState := 220; // Check Completion
663         END_IF;
664
665         //----- 220. Check Completion
-----
666         220:
667         IF loopIndex >= PartsTotal THEN
668             RobotState := 10; // Main Loop (Idle)
669         ELSE
670             RobotState := 30; // Step-Thru Main Loop
671         END_IF;
672
673     END_CASE;
674
675     ELSE
676         AutoMode := FALSE;
677         ConveyorRun_Auto := FALSE;
678         TurnOnElectromagnet_Auto := FALSE;
679         RobotState := 0;
680         VisionFrameID := 0;
681         WriteIdx := 0;
682         ReadIdx := 0;
683         FrameCount := 0;
684     END_IF;
685
```

Function Block

```

1  base_y := -MyRobot.Base_Radius;
2
3  CA1 (
4  Robot   := MyRobot,
5  x_0     := x_0,
6  y_0     := y_0,
7  z_0     := z_0,
8  base_y  := base_y
9  );
10
11 IF NOT CA1.Ok THEN
12     InverseKinematics_Ok := FALSE;
13     RETURN;
14 END_IF;
15
16 theta1 := CA1.theta;
17
18 x_1 := x_0 * MyRobot.cos120 + y_0 * MyRobot.sin120;
19 y_1 := y_0 * MyRobot.cos120 - x_0 * MyRobot.sin120;
20 base_y := -MyRobot.Base_Radius; // * MyRobot.cos120;
21
22 CA2 (
23 Robot   := MyRobot,
24 x_0     := x_1,
25 y_0     := y_1,
26 z_0     := z_0,
27 base_y  := base_y
28 );
29
30 IF NOT CA2.Ok THEN
31     InverseKinematics_Ok := FALSE;
32     RETURN;
33 END_IF;
34
35 theta2 := CA2.theta;
36
37 x_2 := x_0 * MyRobot.cos120 - y_0 * MyRobot.sin120;
38 y_2 := y_0 * MyRobot.cos120 + x_0 * MyRobot.sin120;
39 base_y := -MyRobot.Base_Radius; // * MyRobot.cos120;
40
41 CA3 (
42 Robot   := MyRobot,
43 x_0     := x_2,
44 y_0     := y_2,
45 z_0     := z_0,
46 base_y  := base_y

```

Function Block

```
47 );  
48  
49 IF NOT CA3.Ok THEN  
50     InverseKinematics_Ok := FALSE;  
51     RETURN;  
52 END_IF;  
53  
54 theta3 := CA3.theta;  
55  
56 theta1 := theta1 * 180.0 / PI;  
57 theta2 := theta2 * 180.0 / PI;  
58 theta3 := theta3 * 180.0 / PI;  
59  
60 InverseKinematics_Ok := TRUE;  
61
```


Data Name : AngleToSteps

Function Block

```
1 AngleToSteps := REAL_TO_DINT((angle_deg / 360.0) * steps_per_rev *  
  gear_ratio);
```

Function Block

```

1 IF z_0 = 0.0 THEN
2     Ok := FALSE;
3     RETURN;
4 END_IF;
5
6 y_1 := base_y;
7 y_0_local := y_0 - Robot.t_1;
8
9 a_1 :=
10 (x_0*x_0 + y_0_local*y_0_local + z_0*z_0
11 + Robot.Bicep_Length*Robot.Bicep_Length
12 - Robot.Forearm_Length*Robot.Forearm_Length
13 - y_1*y_1)
14 / (2.0 * z_0);
15
16 b_1 := (y_1 - y_0_local) / z_0;
17
18 d_1 :=
19 -(a_1 + b_1*y_1)*(a_1 + b_1*y_1)
20 + Robot.Bicep_Length*Robot.Bicep_Length*(b_1*b_1 + 1.0);
21
22 IF d_1 < 0.0 THEN
23     Ok := FALSE;
24     RETURN;
25 END_IF;
26
27 y_j := (y_1 - a_1*b_1 - SQRT(d_1)) / (b_1*b_1 + 1.0);
28 z_j := a_1 + b_1*y_j;
29
30 IF (y_1 - y_j) = 0.0 THEN
31     Ok := FALSE;
32     RETURN;
33 END_IF;
34
35 theta := ATAN(-z_j / (y_1 - y_j));
36
37 //IF y_j > y_1 THEN
38 //    theta := theta + PI;
39 //END_IF;
40
41 Ok := TRUE;
42

```

Function Block

		1	2	3	4	5	6	7	8	9	10	11	12
1	(0)	i_bEN Execution command	o_bENO Execution status									SET	o_bENO Execution status
2												RST	o_bOK Normal completion
3											MOV	H0	o_uErrId Error code
4											MOV	H0	o_uStatusId Error code of connection communication error
5												RST	o_bErr Error completion
6											MOV	K1	uConnectionNo [Internal variable] Connection No.
7											MOV	H0FFFF	uCheckChangeOffsetAd... [Internal variable] Check of change offs...
8	(27)	i_bEN Execution command										RST	o_bENO Execution status
9	(31)	bFbRun [Internal variable]FB run										SET	bFbRun [Internal variable]FB run

Function Block

		1	2	3	4	5	6	7	8	9	10	11	12
10	(35)												bFbRun [Internal variable]FB run
11													o_bOK Normal completion
12												H0	o_uErrId Error code
13												H0	o_uStatusId Error code of connection communication error
14													o_bErr Error completion
15													bConnectionNoCheckOK [Internal variable] Connection No. is normal
16												K1	uConnectionNo [Internal variable] Connection No.
17												K0	uInputDataAreaOffset [Internal variable] Offset of input data area
18												K0	uInputDataSize [Internal variable] Input data size

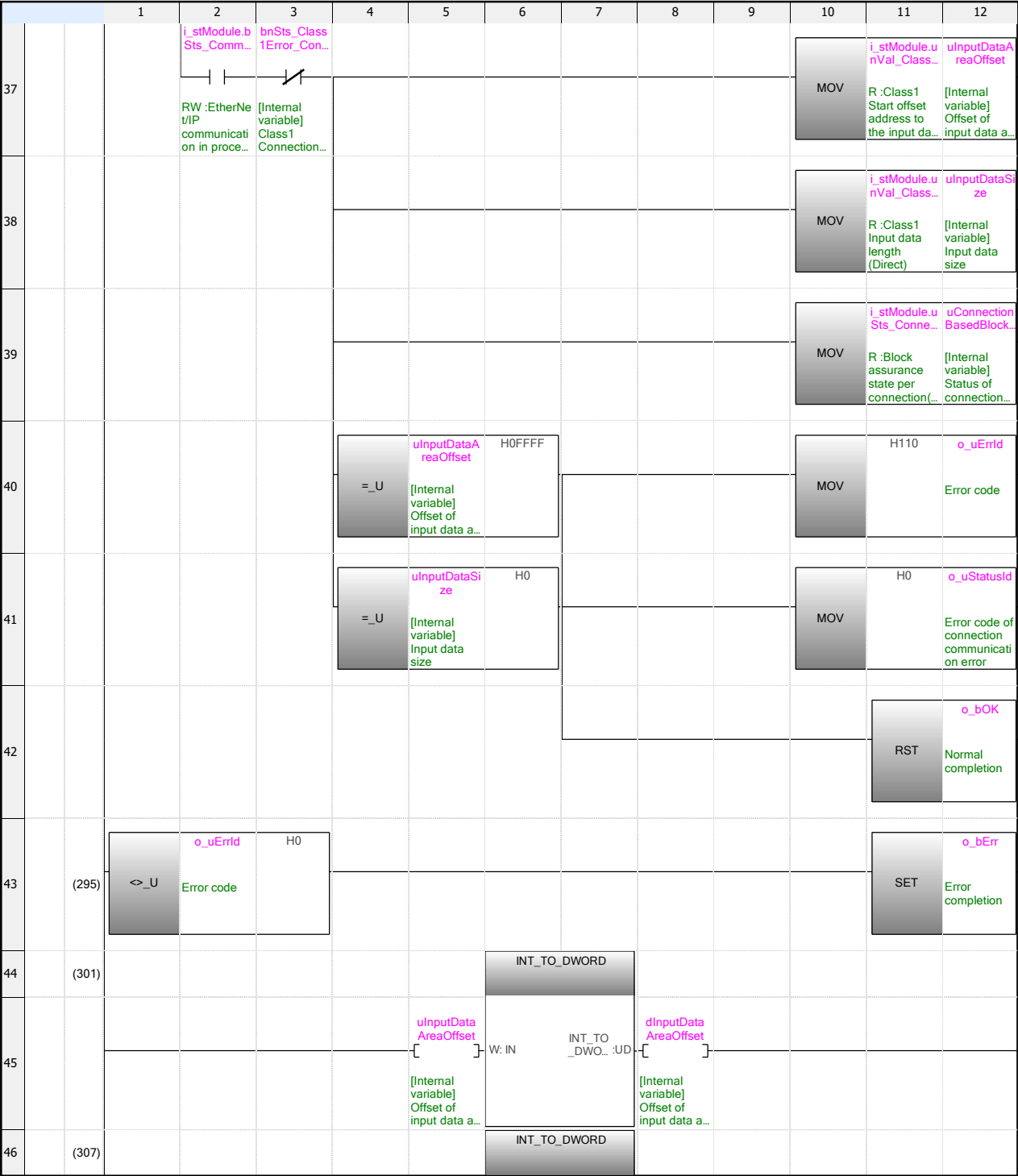
Function Block

	1	2	3	4	5	6	7	8	9	10	11	12
19										MOV	K0	uConnectionBasedBlock... [Internal variable] Status of connection...
20										DMOV	K0	dInputDataAreaOffset [Internal variable] Offset of input data a...
21										DMOV	K0	dInputDataAreaGetAddr... [Internal variable] Get address of input data a...
22										DMOV	K0	dInputDataSize [Internal variable] Input data size (DWO...
23										MOV	H0FFFF	uCheckChangeOffsetAd... [Internal variable] Check of change offs...
24											RST	bChangeInputDataArea... [Internal variable] Change of getting inpu...
25	(91) [Internal variable]FB run	bFbRun	<=_U K1	i_uConnectionNo Connection number	<=_U K32	i_uConnectionNo Connection number					SET	bConnectionNoCheckOK [Internal variable] Connection No. is normal
26										MOV	i_uConnectionNo Connection number	uConnectionNo [Internal variable] Connection No.
27											RST	o_bErr Error completion

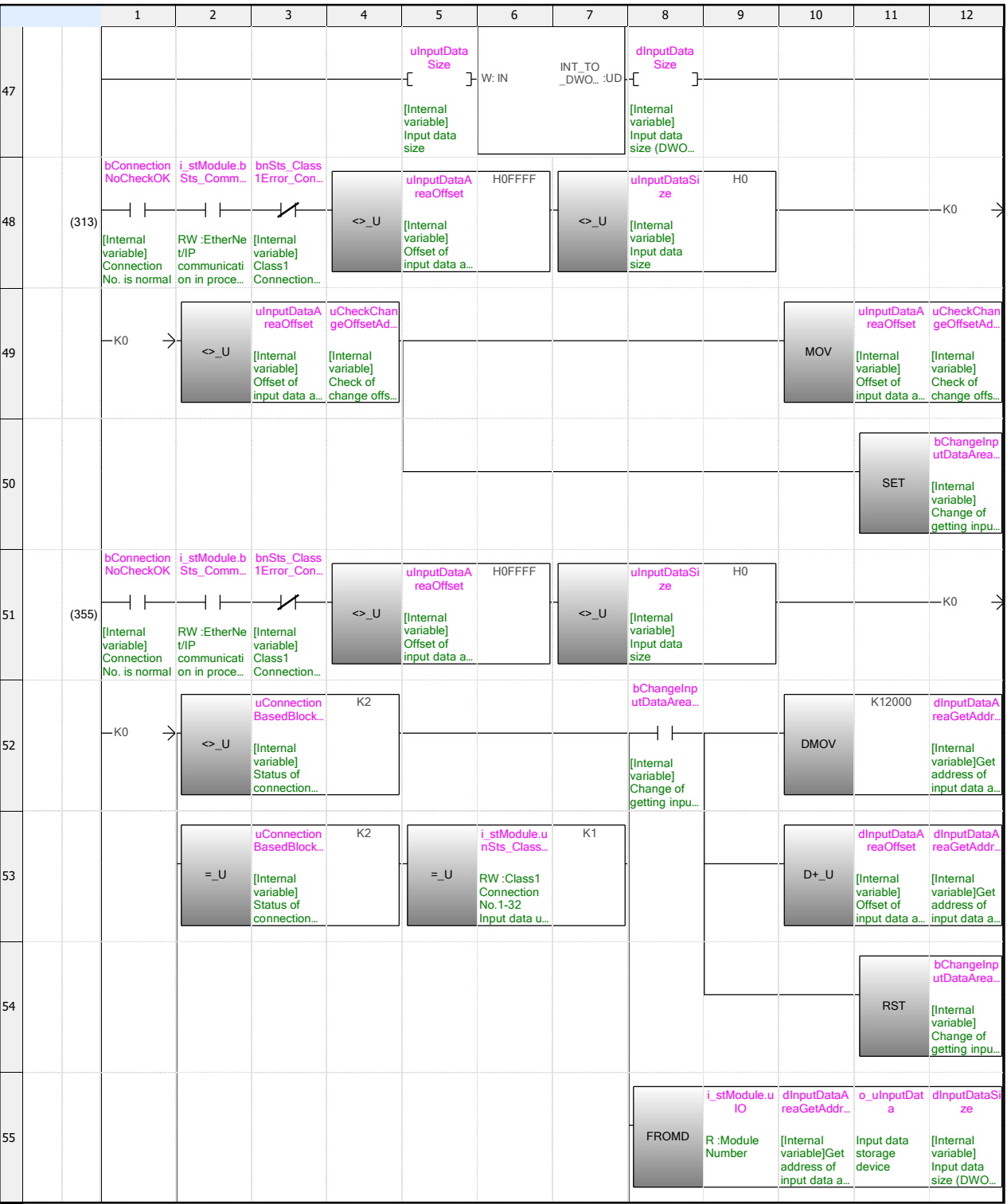
Function Block

		1	2	3	4	5	6	7	8	9	10	11	12
28											MOV	H0	<i>o_uErrId</i> Error code
29											MOV	H0	<i>o_uStatusId</i> Error code of connection communication error
30										BLKMOV	<i>i_stModule.bnSts_Class...</i> R :Class1 Connection No.1-32 Error status...	<i>bnSts_Class1Error_Con...</i> [Internal variable] Class1 Connection...	K64
31			<i>bConnectionNoCheckOK</i> 								MOV	H100	<i>o_uErrId</i> Error code
32			[Internal variable] Connection No. is normal									RST	<i>o_bOK</i> Normal completion
33			<i>bConnectionNoCheckOK</i> 	<i>i_stModule.bnSts_Class1Error_Con...</i> 	<i>bnSts_Class1Error_Con...</i> 						MOV	H200	<i>o_uErrId</i> Error code
34			[Internal variable] Connection No. is normal	RW :EtherNet/IP communication start req...	[Internal variable] Class1 Connection...						MOV	<i>i_stModule.unSts_Class...</i> R :Class1 Connection No.1-32 Behavior er...	<i>o_uStatusId</i> Error code of connection communication error
35												RST	<i>o_bOK</i> Normal completion
36	(188)	<i>bConnectionNoCheckOK</i> 	<i>i_stModule.bnSts_Class1Error_Con...</i> 	<i>bnSts_Class1Error_Con...</i> 	<i>bnSts_Class1Error_Con...</i> 							SET	<i>i_stModule.bnSts_Class1Error_Con...</i> RW :EtherNet/IP communication start req...

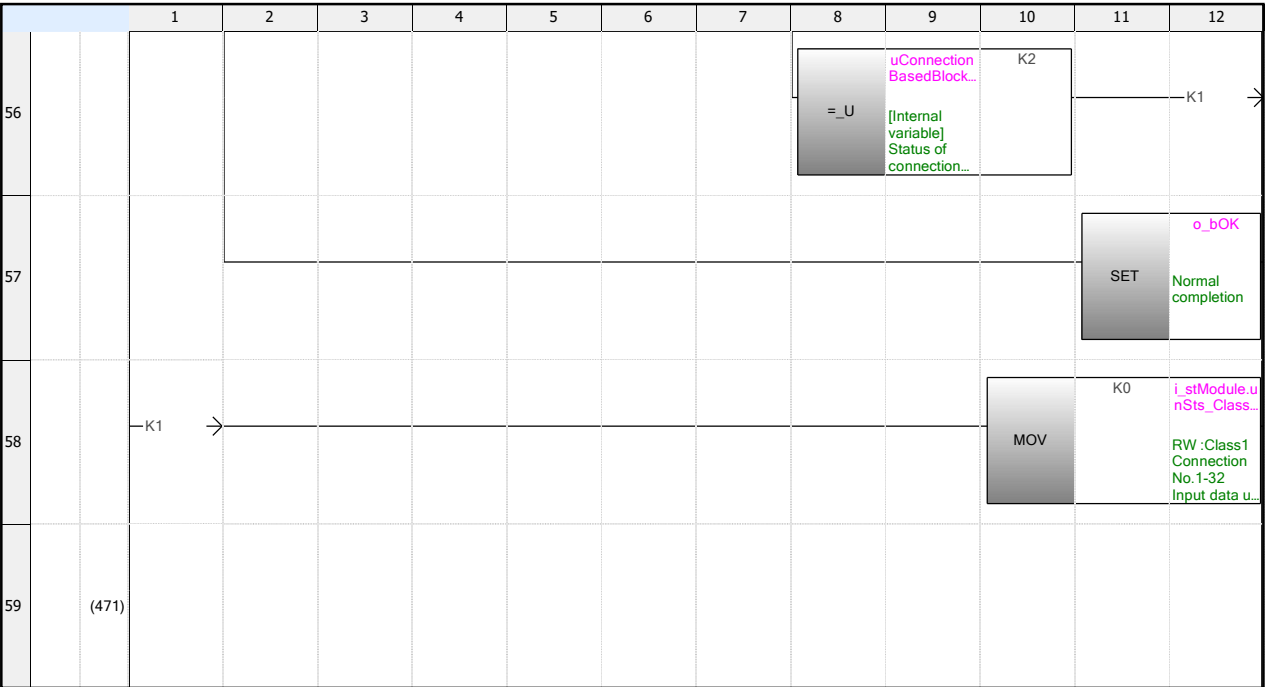
Function Block



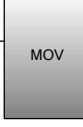
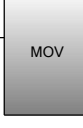
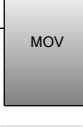
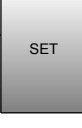
Function Block



Function Block



Function Block

		1	2	3	4	5	6	7	8	9	10	11	12
1	(0)	 Execution command	 Execution status										 Execution status
2													 Normal completion
3												H0	 Error code
4												H0	 Error code of connection communication error
5													 Error completion
6												K1	 [Internal variable] Connection No.
7												H0FFFF	 [Internal variable] Check of change offs...
8	(27)	 Execution command											 Execution status
9	(31)	 [Internal variable]FB run											 [Internal variable]FB run

Function Block

		1	2	3	4	5	6	7	8	9	10	11	12
10	(35)	 Execution command										RST	bFbRun [Internal variable]FB run
11												RST	o_bOK Normal completion
12											MOV	H0	o_uErrId Error code
13											MOV	H0	o_uStatusId Error code of connection communication error
14												RST	o_bErr Error completion
15												RST	bConnectionNoCheckOK [Internal variable] Connection No. is normal
16											MOV	K1	uConnectionNo [Internal variable] Connection No.
17											MOV	K0	uOutputDataAreaOffset [Internal variable] Offset of output data...
18											MOV	K0	uOutputDataSize [Internal variable] Output data size

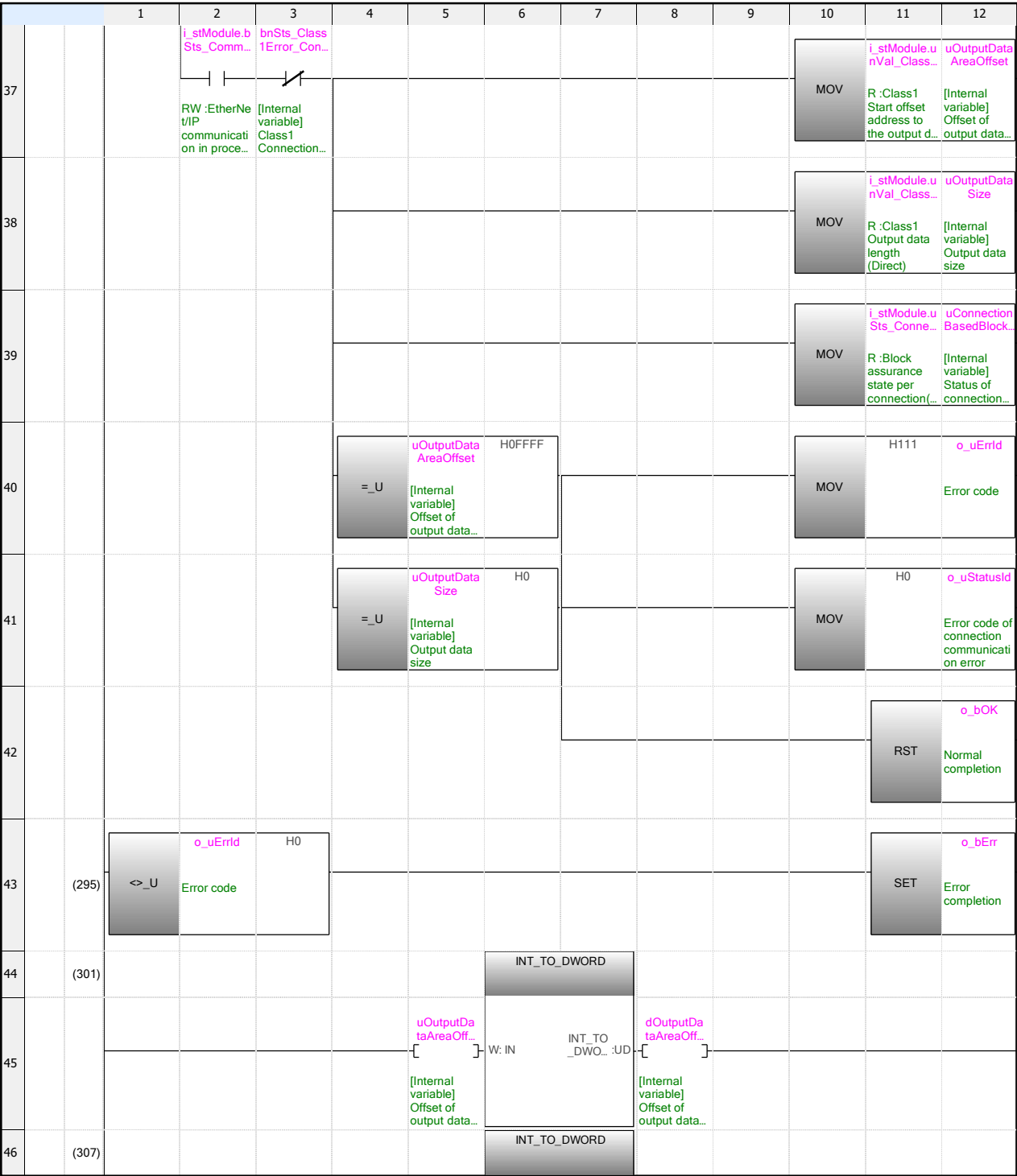
Function Block

		1	2	3	4	5	6	7	8	9	10	11	12
19											MOV	K0	uConnectionBasedBlock... [Internal variable] Status of connection...
20											DMOV	K0	dOutputDataAreaOffset [Internal variable] Offset of output data...
21											DMOV	K0	dOutputDataAreaSetAd... [Internal variable]Set address of output data...
22											DMOV	K0	dOutputDataSize [Internal variable] Output data size (DWO...
23											MOV	H0FFFF	uCheckChangeOffsetAd... [Internal variable] Check of change offs...
24												RST	bChangeOutputDataAre... [Internal variable] Change of setting outp...
25	(91)	bFbRun [Internal variable]FB run	<=_U K1	i_uConnectio nNo Connection number	<=_U K32	i_uConnectio nNo Connection number						SET	bConnectionNoCheckOK [Internal variable] Connection No. is normal
26											MOV	i_uConnectio nNo Connection number	uConnectionNo [Internal variable] Connection No.
27												RST	o_bErr Error completion

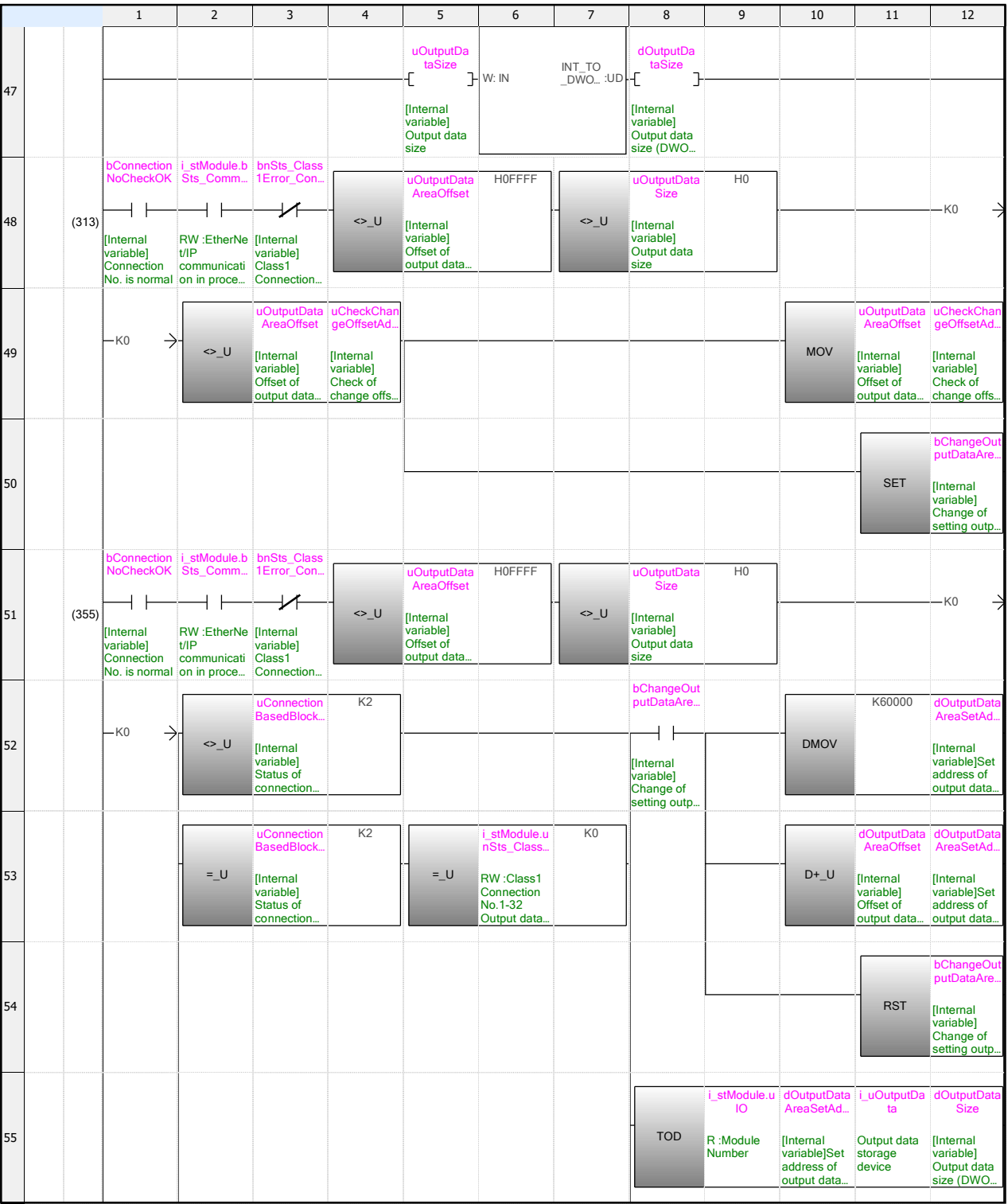
Function Block

		1	2	3	4	5	6	7	8	9	10	11	12
28											MOV	H0	<i>o_uErrId</i> Error code
29											MOV	H0	<i>o_uStatusId</i> Error code of connection communication error
30										BLKMOV	<i>i_stModule.bnSts_Class1Error_Con...</i> R :Class1 Connection No.1-32 Error status...	<i>bnSts_Class1Error_Con...</i> [Internal variable] Class1 Connection...	K64
31			<i>bConnectionNoCheckOK</i> 								MOV	H100	<i>o_uErrId</i> Error code
32			<i>[Internal variable] Connection No. is normal</i>									RST	<i>o_bOK</i> Normal completion
33			<i>bConnectionNoCheckOK</i> 	<i>i_stModule.bnSts_Class1Error_Con...</i> 	<i>RW :EtherNet/IP communication start req...</i> 						MOV	H200	<i>o_uErrId</i> Error code
34			<i>[Internal variable] Connection No. is normal</i>	<i>RW :EtherNet/IP communication start req...</i>	<i>[Internal variable] Class1 Connection...</i>						MOV	<i>i_stModule.bnSts_Class1Error_Con...</i> R :Class1 Connection No.1-32 Behavior er...	<i>o_uStatusId</i> Error code of connection communication error
35												RST	<i>o_bOK</i> Normal completion
36	(188)	<i>bConnectionNoCheckOK</i> 	<i>i_stModule.bnSts_Class1Error_Con...</i> 	<i>i_stModule.bnSts_Class1Error_Con...</i> 	<i>i_stModule.bnSts_Class1Error_Con...</i> 							SET	<i>i_stModule.bnSts_Class1Error_Con...</i> RW :EtherNet/IP communication start req...

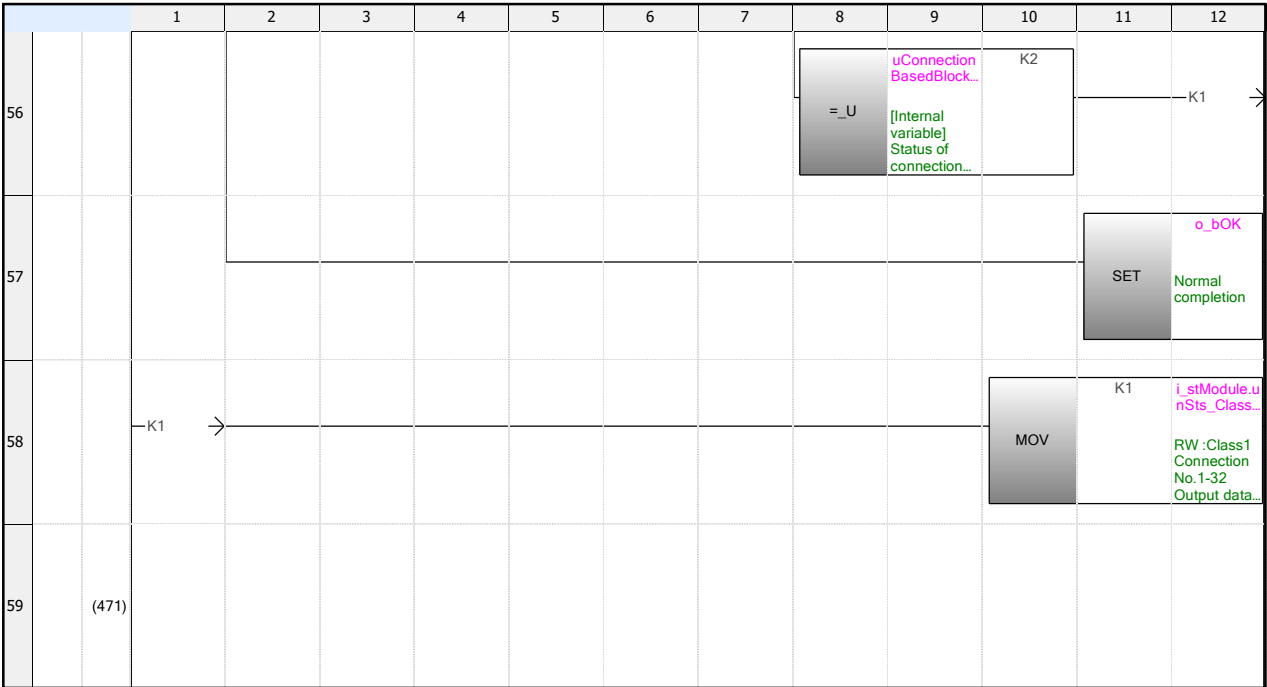
Function Block



Function Block



Function Block



Basic Settings

Item	Setting
 Own Node Settings	
 IP Address	
..... IP Address	10 . 10 . 82 . 1
..... Subnet Mask	255 . 255 . 255 . 0
..... Default Gateway	. . .
..... Communication Data Code	Binary
 CC-Link IEF Basic Settings	
..... To Use or Not to Use CC-Link IEF Basic Setting	Not to Use
..... Network Configuration Settings	<Detailed Setting>
..... Refresh Settings	<Detailed Setting>
 MODBUS/TCP Settings	
..... To Use or Not to Use MODBUS/TCP Setting	Not Used
..... Device Assignment	<Detailed Setting>
 External Device Configuration	
..... External Device Configuration	<Detailed Setting>
 Transmission Port Settings	
..... MELSOFT Transmission Port UDP/IP	Use
..... MELSOFT Transmission Port TCP/IP	Use

Basic Settings
External Device Configuration

No.	Model Name	Communication Measure	Protocol	Fixed Buffer Send/Receive Setting	PLC				Sensor Device				Existence Confirmation
					IP Address	Port No.	MAC Address	Host Name	IP Address	Port No.	Subnet Mask	Default Gateway	
	Host Station				10.10.82.1								
1	MELSOFT Connection Module	MELSOFT Connection	TCP		10.10.82.1								KeepAlive
2	MELSOFT Connection Module	MELSOFT Connection	TCP		10.10.82.1								KeepAlive

Application Settings

Item	Setting
☐ FTP Server Settings	
FTP Server	Not to Use
Login Name	FXCPU
☐ Advanced Settings	
☐ Password Setting	
Current Password	
New Password	
Confirm New Password	
☐ FTP Timer Settings	
Command Input Monitoring Timer	900
Unit	s
Response Monitoring Timer	5
Unit	s
Allow Online Change	Disable
☐ FTP Client Settings	
To Use or Not to Use FTP Client Settings	Not to Use
☐ FTP Server Specification	FTP Server Address
FTP Server IP Address	0 . 0 . 0 . 1
Login Name	
Password	
Connection Method	PORT Mode
Port No.	21
☐ Web Server Settings	
To Use or Not to Use Web Server Settings	Not to Use
Host Station Port No.	80
Account Settings	<Detailed Setting>
☐ Simple CPU Communication Setting	
To Use or Not to Use Simple CPU Communication	Not to Use
Simple CPU Communication Setting	<Detailed Setting>
☐ Time Setting	
Time Setting (SNTP Client)	Not Use
SNTP Server IP Address	0 . 0 . 0 . 1
Timer Setting After Power-on and Reset	Disable
☐ Execution Timing	Fixed Time
Time Intervals	1 Minute
☐ Specified Time (Hour, Minute, Day of Week)	
☐ Clock Time (Hour, Minute)	
Hour	12
Minute	0
☐ Day of Week	
Sun	Set
Mon	Set

Application Settings

Item	Setting
..... Tue	Set
..... Wed	Set
..... Thu	Set
..... Fri	Set
..... Sat	Set
 Security	
.....  IP Filter Settings	
..... IP Filter	Disable
..... IP Filter Settings	<Detailed Setting>
 Disable Direct Connection with MELSOFT	
..... Disable Direct Connection with MELSOFT	Enable
 Do Not Respond to CPU Module Search	
..... Do Not Respond to CPU Module Search	Respond
 iQ Sensor Solution Settings	
..... Use iQ Sensor Solution	Use
 SLMPSEND Instruction Settings	
..... Use SLMPSEND Instruction	Use

Module Parameter : Required Settings

Item	Setting
 Mode	
..... Communication Mode	Online

Module Parameter : Basic Settings

Item	Setting
Own Node Settings	
IP Address	
IP Address	10 . 10 . 82 . 2
Subnet Mask	255 . 255 . 255 . 0
Default Gateway	10 . 10 . 82 . 1
Communication Data Code	Binary
Opening Method	Do Not Open by Program
Operational Setting	
Timer Settings for Data Communication	
Change/Set Timer Value	No
Destination Alive Check Start Interval Timer	600
Unit	s
Destination Alive Check Interval Timer	10
Unit	s
Destination Alive Check Resend Count	3 Times
BACnet Function Setting	
To Use or Not to Use BACnet Function	Not Used
Network Information Setting	
Network No.	0
BACnet Device Setting	
Instance No.	0
Object Name	
BACnet Standard	ANSI/ASHRAE Standard 135-2016
I-Am Send Setting	
Send Control	Send only When Device Status is Operational
Send Setting upon Device Status Change to Operational	Send
Enable/Disable Fixed Interval Send	Disable
Cycle of Fixed Interval Send [Second]	60 Second
I-Am Response Setting	LocalBroadcast
I-Am Send Setting	LocalBroadcast
Device Restart Procedures Setting	
Device Restart Procedures	Not to Support
Report Destination Network No.	0
Report Destination IP Address	192 . 168 . 0 . 254
Report Destination Port No.	47808
Time Synchronization Setting	Ignore
BACnet Object Setting	
Accumulator Object Setting	<Detailed Setting>
AnalogInput Object Setting	<Detailed Setting>
AnalogOutput Object Setting	<Detailed Setting>
AnalogValue Object Setting	<Detailed Setting>
BinaryInput Object Setting	<Detailed Setting>

Module Parameter : Basic Settings

Item	Setting
BinaryOutput Object Setting	<Detailed Setting>
BinaryValue Object Setting	<Detailed Setting>
Multi-state Input Object Setting	<Detailed Setting>
Multi-state Output Object Setting	<Detailed Setting>
External Device Configuration	
External Device Configuration	<Detailed Setting>

Module Parameter : Application Settings

Item	Setting
[-] Simple CPU Communication Setting	
To Use or Not to Use Simple CPU Communication	Not to Use
[-] CPU Response Monitoring Timer	5
Unit	s
Simple CPU Communication Setting	<Detailed Setting>
[-] Security	
[-] IP Filter Settings	
IP Filter	Disable
IP Filter Settings	<Detailed Setting>
[-] Disable Direct Connection with MELSOFT	
Disable Direct Connection with MELSOFT	Enable
[-] Do Not Respond to CPU Module Search	
Do Not Respond to CPU Module Search	Respond

Workspace Name :
Project Name :
Title :

Data Name	Last Change	Title
 Project	4/30/2026 12:13:14 PM	
..... Module Configuration		
 Program		
..... Initial		
 Scan		
..... MAIN	4/30/2026 12:13:14 PM	
..... EIP_Read	4/30/2026 12:06:38 PM	
..... Local Label	4/10/2026 7:14:08 PM	
..... ProgramBody	4/30/2026 12:06:38 PM	
..... EIP_Write	4/30/2026 12:06:38 PM	
..... Local Label	4/10/2026 7:14:13 PM	
..... ProgramBody	4/30/2026 12:06:38 PM	
..... Vision	4/30/2026 12:06:38 PM	
..... Local Label	3/31/2026 10:21:04 AM	
..... ProgramBody	4/30/2026 12:06:38 PM	
..... Motion	4/30/2026 12:06:38 PM	
..... Local Label	3/19/2026 12:32:46 PM	
..... ProgramBody	4/30/2026 12:06:38 PM	
..... Conveyor	4/30/2026 12:06:38 PM	
..... Local Label	1/31/2026 11:42:08 PM	
..... ProgramBody	4/30/2026 12:06:38 PM	
..... Auto	4/30/2026 12:13:14 PM	
..... Local Label	4/14/2026 11:49:26 AM	
..... ProgramBody	4/30/2026 12:13:14 PM	
..... Fixed Scan		
..... Event		
..... Standby		
..... No Execution Type		
..... Unregistered Program		
 FB/FUN	4/30/2026 12:06:37 PM	
..... InverseKinematics	4/30/2026 12:06:36 PM	
..... InverseKinematics	4/30/2026 12:06:36 PM	
..... Local Label	12/29/2025 12:28:24 AM	
..... ProgramBody	4/30/2026 12:06:36 PM	
..... AngleToSteps	4/30/2026 12:06:36 PM	
..... AngleToSteps	4/30/2026 12:06:36 PM	
..... Local Label	12/29/2025 9:05:00 PM	
..... ProgramBody	4/30/2026 12:06:36 PM	

Workspace Name :
Project Name :
Title :

Data Name	Last Change	Title
CalculateAngle_FB	4/30/2026 12:06:36 PM	
CalculateAngle_FB	4/30/2026 12:06:36 PM	
Local Label	12/29/2025 12:35:11 AM	
ProgramBody	4/30/2026 12:06:36 PM	
M_FBLIB	4/30/2026 12:06:37 PM	
M+FX5ENETIP_Class1GetInputData_01A	4/30/2026 12:06:37 PM	Get Class1 input data
Local Label	11/6/2025 1:01:11 PM	
ProgramBody	4/30/2026 12:06:37 PM	
M+FX5ENETIP_Class1SetOutputData_01A	4/30/2026 12:06:37 PM	Set Class1 Output data
Local Label	11/6/2025 1:07:51 PM	
ProgramBody	4/30/2026 12:06:37 PM	
Label		
Global Label	4/26/2026 7:26:10 PM	
VisionTags	4/26/2026 7:26:10 PM	
Main	4/13/2026 9:57:54 AM	
M+Global	11/6/2025 12:44:08 PM	
MotionTags	4/10/2026 8:26:11 PM	
ConveyorTags	2/19/2026 10:50:45 AM	
Structured Data Types	4/10/2026 7:15:43 PM	
M+FX5ENETIP	3/2/2021 7:17:41 PM	
M+FX5UCPU_CPU_Restoration_Function_Setting	7/5/2019 4:10:47 AM	
M+FX5UCPU_CPU_Restoration_Target_Date_Stamp_Folder_Setting	7/5/2019 4:10:56 AM	
M+FX5UCPU_Clock_Data	7/5/2019 4:10:22 AM	
M+FX5UCPU_Error_Information	7/5/2019 4:10:17 AM	
M+FX5UCPU_F	7/5/2019 4:10:12 AM	
M+FX5UCPU_Error_Time	7/5/2019 4:10:06 AM	
M+FX5UCPU_Latest_Error	6/4/2021 12:22:56 AM	
M+FX5UCPU_MemoryCard_Use_Condition	7/5/2019 4:10:39 AM	
M+FX5UCPU_SD_FX5_4AD_AD_P_1	6/4/2021 2:38:23 AM	
M+FX5UCPU_SD_FX5_4AD_AD_P_2	6/4/2021 2:39:12 AM	
M+FX5UCPU_SD_FX5_4AD_AD_P_3	6/4/2021 2:39:50 AM	
M+FX5UCPU_SD_FX5_4AD_AD_P_4	6/4/2021 2:40:37 AM	
M+FX5UCPU_SD_FX5_4AD_PT_AD_P_1	7/5/2019 4:12:59 AM	
M+FX5UCPU_SD_FX5_4AD_PT_AD_P_2	7/5/2019 4:13:24 AM	
M+FX5UCPU_SD_FX5_4AD_PT_AD_P_3	7/5/2019 4:13:48 AM	
M+FX5UCPU_SD_FX5_4AD_PT_AD_P_4	7/5/2019 4:14:17 AM	

Workspace Name :
Project Name :
Title :

Data Name	Last Change	Title
M+FX5UCPU_SD_FX5_4AD_TC_ADP_1	7/5/2019 4:12:53 AM	
M+FX5UCPU_SD_FX5_4AD_TC_ADP_2	7/5/2019 4:13:18 AM	
M+FX5UCPU_SD_FX5_4AD_TC_ADP_3	7/5/2019 4:13:42 AM	
M+FX5UCPU_SD_FX5_4AD_TC_ADP_4	7/5/2019 4:14:10 AM	
M+FX5UCPU_SD_FX5_4A_ADP_1_AD	6/4/2021 2:38:46 AM	
M+FX5UCPU_SD_FX5_4A_ADP_1_DA	6/4/2021 2:39:03 AM	
M+FX5UCPU_SD_FX5_4A_ADP_2_AD	6/4/2021 2:39:37 AM	
M+FX5UCPU_SD_FX5_4A_ADP_2_DA	6/4/2021 2:39:44 AM	
M+FX5UCPU_SD_FX5_4A_ADP_3_AD	6/4/2021 2:40:17 AM	
M+FX5UCPU_SD_FX5_4A_ADP_3_DA	6/4/2021 2:40:24 AM	
M+FX5UCPU_SD_FX5_4A_ADP_4_AD	6/4/2021 2:41:04 AM	
M+FX5UCPU_SD_FX5_4A_ADP_4_DA	6/4/2021 2:41:11 AM	
M+FX5UCPU_SD_FX5_4DA_ADP_1	6/4/2021 2:38:36 AM	
M+FX5UCPU_SD_FX5_4DA_ADP_2	6/4/2021 2:39:24 AM	
M+FX5UCPU_SD_FX5_4DA_ADP_3	6/4/2021 2:40:09 AM	
M+FX5UCPU_SD_FX5_4DA_ADP_4	6/4/2021 2:40:52 AM	
M+FX5UCPU_SD_Analog_Adapter	6/4/2021 2:38:15 AM	
M+FX5UCPU_SD_Analog_Input	6/4/2021 2:37:57 AM	
M+FX5UCPU_SD_Analog_Output	6/4/2021 2:38:05 AM	
M+FX5UCPU_SD_CPU_Module_Built_In_Analog	6/4/2021 2:37:47 AM	
M+FX5UCPU_SD_DataLogging_Setting	7/5/2019 4:11:01 AM	
M+FX5UCPU_SD_Data_Logging_Function	2/25/2021 10:16:21 PM	
M+FX5UCPU_SD_FX_Compatible_Area	2/6/2023 6:15:26 PM	
M+FX5UCPU_Module_Information	7/5/2019 4:11:16 AM	
M+FX5UCPU_SD_FX_Dedicated	6/4/2021 12:22:56 AM	
M+FX5UCPU_SD_HighSpeedComparison	7/5/2019 4:11:33 AM	
M+FX5UCPU_SD_HighSpeedCounter	7/5/2019 4:11:28 AM	
M+FX5UCPU_SD_PWM	7/5/2019 4:11:45 AM	
M+FX5UCPU_SD_PulseWidthMeasurement	7/5/2019 4:11:38 AM	
M+FX5UCPU_SD_HighSpeedInOut	6/4/2021 12:22:56 AM	
M+FX5UCPU_SD_Latch_Area	2/25/2021 10:16:25 PM	
M+FX5UCPU_SD_Position	7/5/2019 4:12:03 AM	
M+FX5UCPU_SD_Inverter	2/25/2021 10:08:05 PM	
M+FX5UCPU_SD_MC_Protocol	2/25/2021 10:05:27 PM	
M+FX5UCPU_SD_MODBUS	2/25/2021 10:05:31 PM	
M+FX5UCPU_SD_NN_Network	2/25/2021 10:08:09 PM	
M+FX5UCPU_SD_Parallel_Link	2/25/2021 10:11:13 PM	

Workspace Name :
Project Name :
Title :

Data Name	Last Change	Title
M+FX5UCPU_SD_Predefined_Protocol	2/25/2021 10:13:58 PM	
M+FX5UCPU_SD_SerialComInfo	7/5/2019 4:14:39 AM	
M+FX5UCPU_SD_SerialComMon	7/5/2019 4:14:47 AM	
M+FX5UCPU_SD_SerialComSettingInfo	7/5/2019 4:14:54 AM	
M+FX5UCPU_SD_SerialCom	6/4/2021 12:22:56 AM	
M+FX5UCPU_Time	7/5/2019 4:10:32 AM	
M+FX5UCPU_Scan_Time	6/4/2021 12:22:56 AM	
M+FX5UCPU_SD	2/6/2023 6:15:15 PM	
M+FX5UCPU_SD_Diagnostics1	7/5/2019 4:16:26 AM	
M+FX5UCPU_SD_Diagnostics2	7/5/2019 4:16:32 AM	
M+FX5UCPU_SD_LinkScanTime	7/5/2019 4:16:21 AM	
M+FX5UCPU_SD_Basic	7/26/2022 7:18:33 PM	
M+FX5UCPU_SD_PredefinedProtocol	7/5/2019 4:16:08 AM	
M+FX5UCPU_SD_Time_SynchronizeTimeExecution	2/25/2021 10:21:00 PM	
M+FX5UCPU_SD_EtherPort	6/4/2021 12:22:56 AM	
M+FX5UCPU_SD_SimplePLC	1/21/2022 12:00:42 AM	
M+FX5UCPU_Data_Backup_Restoration_Function	7/5/2019 4:09:16 AM	
M+FX5UCPU_Extended_File_Register_Function	7/5/2019 4:09:27 AM	
M+FX5UCPU_Firmware_Update_Function	7/5/2019 4:05:18 AM	
M+FX5UCPU_Latch_Area	7/5/2019 4:09:21 AM	
M+FX5UCPU_SM_FX5_4AD_ADP_1	6/4/2021 2:31:05 AM	
M+FX5UCPU_SM_FX5_4AD_ADP_2	6/4/2021 2:32:46 AM	
M+FX5UCPU_SM_FX5_4AD_ADP_3	6/4/2021 2:35:39 AM	
M+FX5UCPU_SM_FX5_4AD_ADP_4	6/4/2021 2:36:31 AM	
M+FX5UCPU_SM_FX5_4AD_PT_ADP_1	7/5/2019 4:07:21 AM	
M+FX5UCPU_SM_FX5_4AD_PT_ADP_2	7/5/2019 4:07:46 AM	
M+FX5UCPU_SM_FX5_4AD_PT_ADP_3	7/5/2019 4:08:10 AM	
M+FX5UCPU_SM_FX5_4AD_PT_ADP_4	7/5/2019 4:08:42 AM	
M+FX5UCPU_SM_FX5_4AD_TC_ADP_1	7/5/2019 4:07:15 AM	
M+FX5UCPU_SM_FX5_4AD_TC_ADP_2	7/5/2019 4:07:40 AM	
M+FX5UCPU_SM_FX5_4AD_TC_ADP_3	7/5/2019 4:08:04 AM	
M+FX5UCPU_SM_FX5_4AD_TC_ADP_4	7/5/2019 4:08:36 AM	
M+FX5UCPU_SM_FX5_4A_ADP_1_AD	6/4/2021 2:32:24 AM	
M+FX5UCPU_SM_FX5_4A_ADP_1_DA	2/25/2021 7:10:17 PM	
M+FX5UCPU_SM_FX5_4A_ADP_2_AD	6/4/2021 2:35:22 AM	
M+FX5UCPU_SM_FX5_4A_ADP_2_DA	2/25/2021 7:19:26 PM	
M+FX5UCPU_SM_FX5_4A_ADP_3_AD	6/4/2021 2:36:13 AM	

Workspace Name :
Project Name :
Title :

Data Name	Last Change	Title
M+FX5UCPU_SM_FX5_4A_ADP_3_DA	2/25/2021 7:23:35 PM	
M+FX5UCPU_SM_FX5_4A_ADP_4_AD	6/4/2021 2:36:54 AM	
M+FX5UCPU_SM_FX5_4A_ADP_4_DA	2/25/2021 7:27:46 PM	
M+FX5UCPU_SM_FX5_4DA_ADP_1	7/5/2019 4:07:28 AM	
M+FX5UCPU_SM_FX5_4DA_ADP_2	7/5/2019 4:07:52 AM	
M+FX5UCPU_SM_FX5_4DA_ADP_3	7/5/2019 4:08:16 AM	
M+FX5UCPU_SM_FX5_4DA_ADP_4	6/4/2021 12:22:58 AM	
M+FX5UCPU_SM_Analog_Adapter	6/4/2021 2:30:52 AM	
M+FX5UCPU_SM_Analog_Input	6/4/2021 2:30:33 AM	
M+FX5UCPU_SM_Analog_Output	7/5/2019 4:06:54 AM	
M+FX5UCPU_SM_CPU_Module_Built_In_Analog	6/4/2021 2:30:19 AM	
M+FX5UCPU_SM_DataLogging_Setting	7/5/2019 4:05:30 AM	
M+FX5UCPU_SM_Data_Logging_Function	7/5/2019 4:09:10 AM	
M+FX5UCPU_SM_FX_Compatible_Area	2/6/2023 6:15:21 PM	
M+FX5UCPU_SM_HighSpeedComparison	7/5/2019 4:05:59 AM	
M+FX5UCPU_SM_HighSpeedCounter	8/23/2019 12:41:45 AM	
M+FX5UCPU_SM_PWM	7/5/2019 4:06:22 AM	
M+FX5UCPU_SM_PulseWidthMeasurement	7/5/2019 4:06:08 AM	
M+FX5UCPU_SM_HighSpeedInOut	6/4/2021 12:22:56 AM	
M+FX5UCPU_SM_Position	7/5/2019 4:06:34 AM	
M+FX5UCPU_SM_Inverter	2/25/2021 7:38:08 PM	
M+FX5UCPU_SM_MC_Protocol	2/25/2021 7:38:00 PM	
M+FX5UCPU_SM_MODBUS	2/25/2021 7:38:04 PM	
M+FX5UCPU_SM_NN_Network	2/25/2021 7:38:12 PM	
M+FX5UCPU_SM_Parallel_Link	2/25/2021 7:38:15 PM	
M+FX5UCPU_SM_SerialComError	5/12/2021 1:28:24 AM	
M+FX5UCPU_SM_SerialComFlag	7/5/2019 4:09:04 AM	
M+FX5UCPU_SM_SerialCom	6/4/2021 12:22:56 AM	
M+FX5UCPU_SM	2/6/2023 6:14:58 PM	
M+FX5UCPU	2/6/2023 6:15:03 PM	
EIP_Cognex_Input	3/31/2026 10:17:39 AM	
EIP_Cognex_Output	3/17/2026 6:42:00 PM	
EIP_Motion_Input	11/15/2025 1:54:27 PM	
EIP_Motion_Output	11/13/2025 10:46:44 PM	
VisionFrame	3/31/2026 10:17:56 AM	
DeltaRobot	1/7/2026 12:21:09 AM	

Workspace Name :
Project Name :
Title :

Data Name	Last Change	Title
..... Position	12/30/2025 9:25:37 PM	
..... PositionBuffer	1/4/2026 2:55:42 PM	
..... Device		
..... Device Comment		
..... Each Program Device Comment		
..... Common Device Comment	4/30/2026 12:06:01 PM	
..... Device Memory	4/30/2026 12:06:40 PM	
..... MAIN	4/30/2026 12:06:40 PM	
..... Device Initial Value	4/30/2026 12:05:51 PM	
..... Parameter		
..... System Parameter		
..... FX5UCPU		
..... CPU Parameter	4/30/2026 12:06:28 PM	
..... Module Parameter		
..... Ethernet Port		
..... 485 Serial Port		
..... High Speed I/O		
..... Input Response Time		
..... Analog Input		
..... Analog Output		
..... Expansion Board		
..... Memory Card Parameter		
..... Module Information		
..... 1[U1]:FX5-ENET/IP	4/30/2026 12:06:33 PM	
..... Module Parameter		
..... Remote Password		

[Line Statement List]

Step	Line Statement
(0)	[Title] Manual Signals
	These are used to prevent having to use SET'S and RESET;s in HMI. My preference. get over it
	HMI buttons toggle states. Lamps use direct Input/Output as verification. So Secondary command bits are local to save comms size
(300)	[Title] Signal Commands
(371)	[Title] Fault Check
	Main Error check
	If any faults present, SET Vision State to ERROR
(492)	Latch Fault Status to prevent unwanted motion upon startup
(631)	[Title] Unit Conversion, Sort, Frame Setting

[Line Statement List]

Step	Line Statement
(0)	[Title] Manual Signals
	These are used to prevent having to use SET'S and RESET;s in HMI. My preference. get over it
(169)	[Title] Signal Commands
(240)	[Title] Fault Check
	Main Error check
	If any faults present, SET Vision State to ERROR
(361)	Latch Fault Status to prevent unwanted motion upon startup
(496)	[Title] Speed Setting
(703)	[Title] Homing Seq
(811)	[Title] Work Envelope Demo

[Line Statement List]

Step	Line Statement
(0)	[Title] Manual Signals
	These are used to prevent having to use SET'S and RESET;s in HMI. My preference. get over it
(142)	To prevent unwanted motion whenn toggling between auto and manual, unless latch is set
(260)	[Title] Signal Commands
(301)	[Title] Fault Checks
	Main Error check
	If any faults present, SET Vision State to ERROR
(423)	Latch Fault Status to prevent unwanted motion upon startup